Fundamentals of Information Systems
for Information Sciences

Second edition

Cinzia Cappiello, Mariagrazia Fugini, Paul Grefen,
Barbara Pernici, Pierluigi Plebani, Monica Vitali

September 18, 2025

DISCLAIMER This book is an English translation of the original and official Italian version. The translation has been carried out using an automatic tool. While every effort has been made to ensure accuracy, there may be errors, inaccuracies, or misinterpretations in the translated text. For authoritative reference, please consult the original Italian edition.

This work is released under the terms of the Creative Commons Attribution-ShareAlike 3.0 Italy license, the text of which is available on the Internet page
`http:creativecommons.org/licenses/by-sa/3.0/it/`

Contents

I Definitions and Characteristics of an IS **1**

1 Introduction **3**
1.1 Data and Information 4
1.2 Organizations 6
1.2.1 Resources 6
1.2.2 Processes 8
1.3 Definition and Role of Information System 10
1.4 Classification of Information Systems 11
1.4.1 OLTP and OLAP Systems 14
1.5 Organization, IT, and Components of an IS 15
1.6 Presentation of the Book Structure 18
1.7 Questions 19

2 Enterprise Architecture **21**
2.1 Enterprise Architecture Framework 21

II Enterprise Modelling **27**

3 Modeling with ArchiMate **29**
3.1 Introduction to ArchiMate 29
3.2 Main Elements of the Model 32
3.2.1 Components 32
3.2.2 Relationships 32
3.2.3 Business layer 33
3.2.4 Application layer 34
3.2.5 Technology layer 36
3.3 Examples 39
3.3.1 Example of representing a document composition system . . 39
3.3.2 Example of an insurance service 39

4 Process Modelling **43**
4.1 Process Modelling in BPMN 43
4.1.1 Definitions 43

4.1.2 Business Process Modeling Notation 44
4.2 Process Identification 47
4.2.1 Sectoral Checklists 49
4.2.2 Porter's Value Chain 50
4.2.3 Analytical Approach 51

III Technological Aspects 53

5 Introduction to Technological Elements 55
5.1 Introduction 55
5.1.1 Application-level technology 55
5.1.2 Platform-level technology 57
5.1.3 Physical architecture-level technology 58
5.2 Application and Technological Components in ArchiMate 59

6 Application Layer: ERP and CRM 61
6.1 Evolution of Application Components 61
6.2 Functional Components 62
6.3 ERP 63
6.3.1 History of ERP 64
6.3.2 Properties of ERPs 65
6.3.3 ERP Functionalities 66
6.3.4 Architectural Aspects 67
6.3.5 ERP Implementation 67
6.3.6 ERP Benefits 69
6.4 CRM 69
6.4.1 Operational CRM 72
6.4.2 Analytical CRM 75
6.4.3 Collaborative CRM 75
6.5 Questions 75

7 Application Layer: Data Analysis 77
7.1 Data Warehousing 77
7.1.1 Data Characteristics 78
7.1.2 Data Warehouse 80
7.1.3 Data Warehouse Architecture 81
7.1.4 Conceptual Model of the Data Warehouse 83
7.1.5 Logical Models of the Data Warehouse 84
7.1.6 Data Warehouse Operations 87
7.1.7 Life Cycle of the Data Warehouse 89
7.2 Data Mining 90
7.2.1 Characterization and Discrimination 91
7.2.2 Association Rules 92
7.2.3 Classificazione 93
7.2.4 Clustering 96

7.2.5 Examples on CRM . . . 98
7.3 Process mining . . . 99
7.3.1 Event Logs . . . 100
7.3.2 Process Discovery . . . 101
7.3.3 Conformance Checking . . . 102
7.3.4 Process Improvement . . . 102
7.4 Questions . . . 103

8 Platform-Level Technologies 105
8.1 Integration Architectures . . . 105
8.2 Application levels and integration techniques . . . 107
8.3 Application Integration Models . . . 110
8.3.1 Point-to-Point Integration Architecture . . . 110
8.3.2 Hub-and-Spoke Integration Architecture . . . 111
8.3.3 Integration with Business Process Management System . . . 112
8.3.4 Service-based Models . . . 113
8.3.5 Service Composition and Orchestration . . . 116
8.4 Questions . . . 117

9 Technologies at the Physical Architecture Level 119
9.1 Description . . . 119
9.1.1 Evolution of Physical Architectures . . . 119
9.2 Distributed Applications . . . 121
9.3 Scalability . . . 126
9.3.1 Server Farm . . . 127
9.4 Virtualization . . . 129
9.5 Cloud Computing . . . 132
9.5.1 Characteristics of Cloud Computing . . . 132
9.5.2 Service Models . . . 133
9.5.3 Deployment Models . . . 133
9.6 Questions . . . 134

10 Sicurezza dei sistemi informativi 135
10.1 Proprietà di sicurezza . . . 135
10.2 Minacce, violazioni, vulnerabilità, attacchi . . . 136
10.3 Crittografia per la protezione dei dati . . . 139
10.3.1 Crittografia simmetrica . . . 140
10.3.2 Crittografia asimmetrica . . . 142
10.3.3 Integrità e funzione di hash . . . 145
10.3.4 Firma digitale . . . 146
10.3.5 Aspetti di gestione delle chiavi e certificati digitali . . . 146
10.4 Gestione degli utenti e controllo degli accessi . . . 149
10.4.1 Autenticazione e autorizzazione . . . 150
10.4.2 Autenticazione a due o più fattori . . . 151
10.4.3 Sistemi di controllo di accesso ai dati . . . 152
10.5 Meccanismi di sicurezza infrastrutturali . . . 161

10.5.1 Firewall 161
10.5.2 Intrusion Detection Systems - IDS 164
10.6 Elementi di cybersecurity 168
10.6.1 Attacchi, vittime, contromisure 169
10.6.2 Zero Trust 170
10.7 Evoluzione 173
10.8 Domande 175

IV Information System Design 177

11 La progettazione di sistemi informativi 179
11.1 Processo di gestione del sistema informativo 179
11.2 Pianificazione 181
11.2.1 Pianificazione strategica 182
11.2.2 Studio di fattibilità 184
11.2.3 Analisi Make-or-Buy 188
11.2.4 Analisi delle strategie di sourcing 189
11.3 Ciclo di vita di sviluppo del SI 192
11.3.1 Ciclo di sviluppo con strategia Make 192
11.3.2 Ciclo di sviluppo con strategia Buy 193
11.4 Domande 197

Part III

Technological Aspects

Chapter 5

Introduction to Technological Elements

5.1 Introduction

The technological components of information systems can be classified according to the categories identified in Figure 5.1.

Considering the architecture of information systems, Figure 5.1 can be seen as a map of the main technologies usable for the actual implementation of these systems. Technologies are divided into three levels: (i) application level, (ii) platform level, and (iii) physical architecture level. The actual selection of technologies to be used at each level is also guided by the application domain considered (*Application Domains* in Figure 5.1). Each application domain is characterized by different goals and needs that inevitably lead to different choices and implementations.

5.1.1 Application-level technology

Application-level technology includes all software applications that can be adopted to support various business functions in operational and informational systems. The set of components at the application level of a company is defined as the *application portfolio*. The portfolio is usually built by selecting some of the following types of applications:

- **DW - Data Warehouse**: archive containing an organization's data, designed using a multidimensional model to facilitate analysis in support of managerial decisions.

- **BI - Business Intelligence**: a set of modules for collecting and analyzing information useful for assessing the current business situation and forecasting future events.

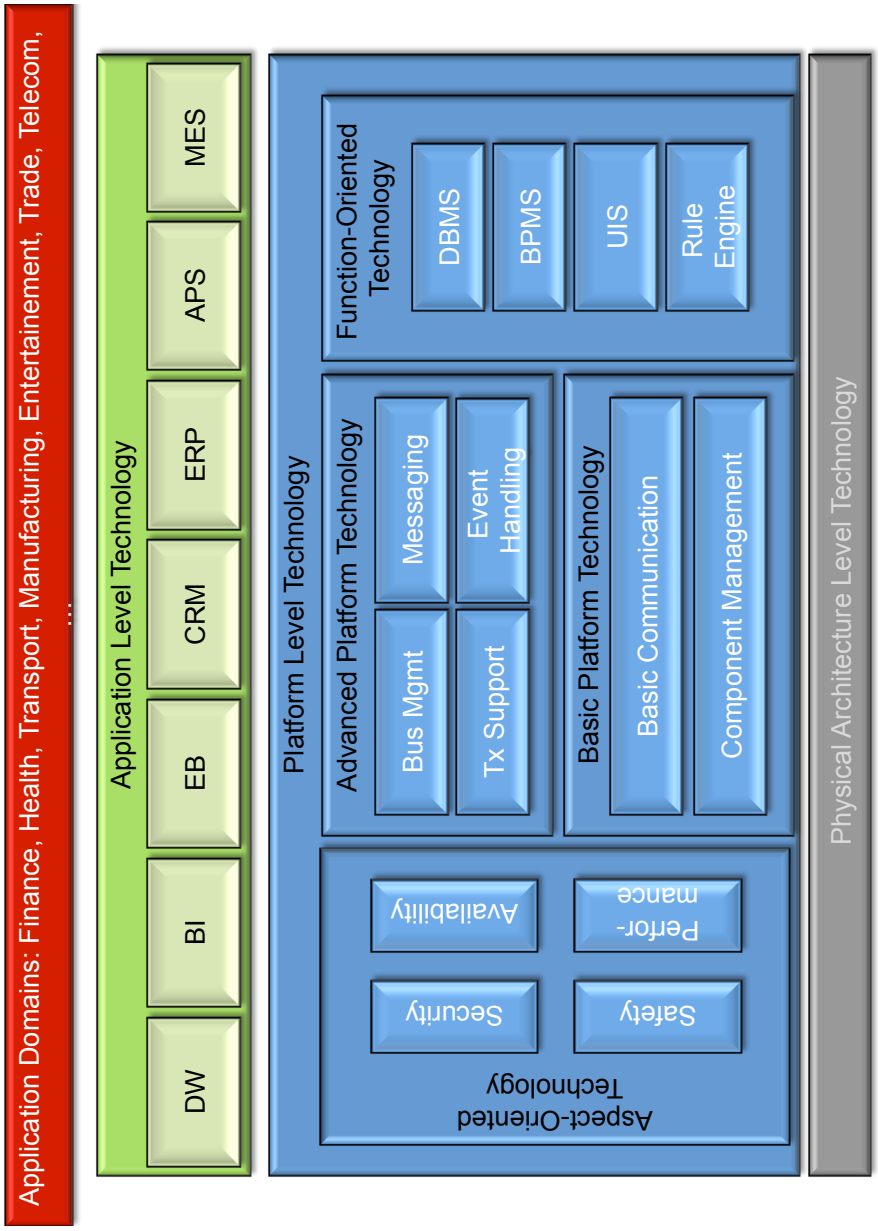


Figure 5.1: Technological components

- **EB - E-Business**: this term refers to applications that support business functions using digital tools for communication, collaboration between companies, and execution of business transactions.

- **CRM - Customer Relationship Management**: a set of modules supporting a strategy for managing interactions with potential and existing customers. These modules are responsible for storing and analyzing customer interactions, and for offering innovative services aimed at customer loyalty.

- **ERP - Enterprise Resource Planning**: software suite composed of modules primarily aimed at supporting a company's operational activities. For example, in a manufacturing company, ERP modules can support all primary and support activities constituting Porter's value chain.

- **APS - Advanced Planning and Scheduling**: set of applications used in manufacturing to optimally manage raw materials and production capacity.

- **MES - Manufacturing Execution Systems**: systems used in manufacturing to track and document the production process from raw materials to finished products.

When deciding which technologies to adopt at the application level, in addition to the type, the best approach for system implementation must be considered, choosing also between Make-or-Buy strategies. For functionalities now widely used across various industries and thus highly standardized, Buy solutions are often used, allowing the adoption of COTS (Commercial Off-The-Shelf) software modules, as is typically the case for ERP suites, based on consolidated and customizable procedures according to business needs. These modules must then be integrated with modules developed using the Make approach to meet the organization's specific requirements (this topic is further explored in Section 11.2.2).

The application modules listed above correspond to application-level components in ArchiMate and may themselves be composed of more detailed functional modules or use services provided by other application components.

5.1.2 Platform-level technology

The implementation of applications requires the use of various platform infrastructure technologies, allowing application components to implement their functionalities by using support components, such as data access, inter-application communication, and security. Technologies at this level can be divided into four categories: (i) Functionality-oriented technologies, (ii) Non-functional-oriented technologies, (iii) Basic technologies, and (iv) Advanced technologies.

Among the **functionality-oriented technologies**, we find:

- **DBMS - Database Management Systems**: systems for managing, controlling, and optimizing data access and manipulation.

- **BPMS - Business Process Management Systems**: systems for modeling, automating, executing, monitoring, and optimizing business activity flows (i.e., processes).

- **UIS - User Interface Systems**: a set of services capable of supporting user interaction and managing different types of interfaces in an adaptive and flexible way, including advanced interaction features and management of graphical and multimedia elements.

- **Rule Engine**: software system that allows defining, testing, and executing business rules at runtime independently from applications.

Non-functional-oriented technologies include technologies that guarantee some non-functional characteristics of applications, such as efficiency, security, or correctness. In Figure 5.1, systems are particularly highlighted for ensuring application and communication security, availability of delivered functionalities, system performance, and safety with respect to physical and environmental risks.

Basic technologies include technologies for managing communication between applications and components. Component management systems allow for monitoring and tracking components and services used within the organization, facilitating both reuse and traceability.

The last category includes **advanced platform technologies** (Advanced platform technology in Figure 5.1). Among these are messaging systems, event and transaction management (Tx support) systems that support integration and communication between applications.

5.1.3 Physical architecture-level technology

As discussed in the introduction, current information systems rely on IT systems that constitute their automated component. IT system architectures have developed and evolved over the years from centralized schemes to distributed models, which better respond to the decentralization and cooperation needs of modern organizations. Technologies at the physical architecture level include paradigms for allocating applications to physical machines, specifying outsourcing solutions such as Cloud Computing. From an architectural point of view, it is important to define who manages the provided infrastructure. It is possible to reason in terms of physical infrastructure management, or at the platform level, or even at the application level. The recent trend is resource virtualization, making them available as *services*. In this case, no physical server will be available, but rather virtual servers (Virtual Machine - VM) with characteristics similar to their physical counterparts. Virtualization of physical resources makes the system more flexible, which becomes a set of virtual resources dynamically corresponding (i.e., changing according to computing needs) to actually available physical resources.

Details on technologies related to physical architectures will be described in later chapters.

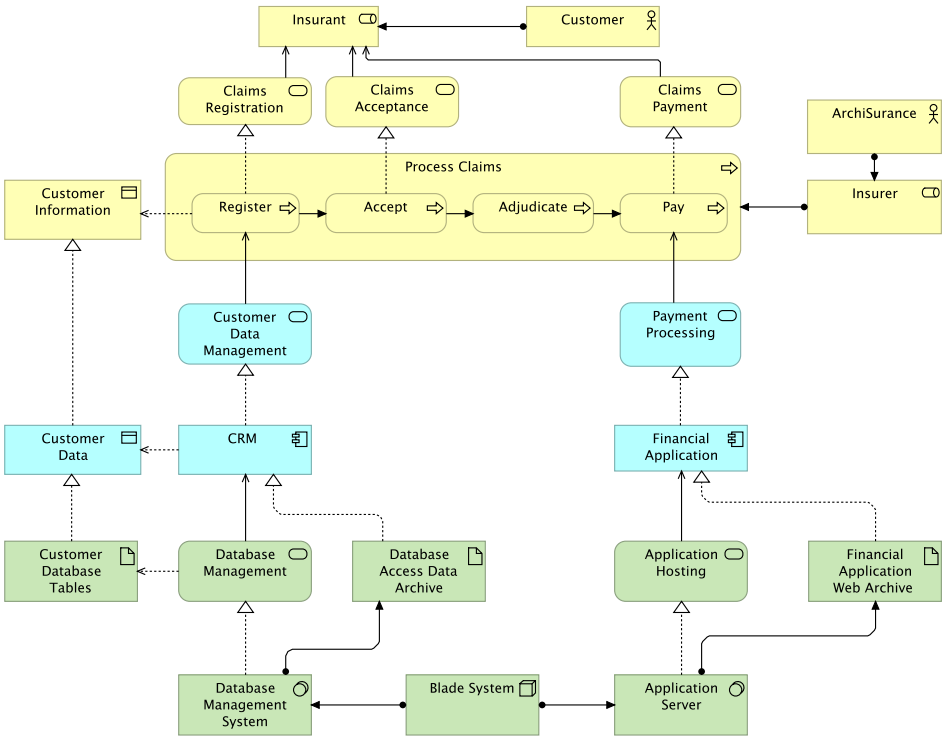


Figure 5.2: Example of component representation across the three layers in ArchiMate (source: The OpenGroup, 2019)

5.2 Application and Technological Components in ArchiMate

Figure 5.2 illustrates an example of the implementation of business-level services using application and technological components with the ArchiMate notation.

At the business level, three services are provided to the insured party (Insurant: the insured is a role, assumed by a Customer, who is an actor, in this case a person): claim registration, claim acceptance, and claim payment. The process that supports claims management is Process Claims, which includes the Register process implementing Claims Registration, Accept implementing Claims Acceptance, and Pay implementing Claims Payment. Process Claims is assigned to the Insurer role performed by the company ArchiSurance.

At the application level, two services—Customer Data Management and Payment Processing—serve the Register and Pay processes.

Customer Data Management is implemented at the application level by a CRM system managing customer data. At the technology level, it uses a platform service for database management (Database Management), which accesses customer data

tables (Customer Database Tables). The Database Management System (DBMS) is a system-level software on the platform that actively executes the Database Access Java Archive object that handles data access. The DBMS runs on a node (Blade System) as an active element, constituting the physical architecture of this system. No additional implementation details of the CRM application are shown in the figure.

The Payment Processing service is implemented by a Financial Application Web Archive object, which realizes the Financial Application.

An Application Hosting service supports both the CRM system and the Financial Application and is implemented by an Application Server system software, assigned to the Blade System node.

Chapter 6

Main Components at the Application Layer

In this chapter, we will analyze in more detail the application-level technologies introduced in Chapter 5. It has already been discussed how the information system must support various business processes that allow for the execution of specific functionalities. These processes can be fully or partially automated through software applications. In this chapter, we introduce the evolution of application components and the main applications that make up the operational-level portfolio of an organization. In Chapter 7, we will instead introduce the main application components that support the tactical and strategic level of Anthony's pyramid.

6.1 Evolution of Application Components

The application components of information systems have undergone significant evolution over time. The first applications were developed using languages close to machine language (assembler or similar) and encoded only basic functionalities. In the 1970s, operating systems began to allow the separation of application logic from hardware interface programming. Each application was independent and used dedicated data saved in files. In the 1980s, DBMSs emerged, enabling data sharing between different applications, avoiding redundancy and decoupling the physical structure of data from the way applications accessed them. Finally, graphical interfaces were introduced to simplify user interaction with applications, replacing previous textual interfaces.

An application component can be broken down into different elements, both from a functional and architectural point of view. The architecture of an application can be very complex. However, in general, applications can be structured into three logical layers, often referred to in the literature as *layers*, which correspond to three software layers into which the applications are partitioned:

- The *presentation layer (P)* is responsible for managing presentation logic, that

is, the methods of interaction with the user. It includes the graphical interface and information rendering. This layer is sometimes also referred to as the *front end* of the application and is accessible to users via a GUI (Graphical User Interface) or UI. This layer is usually web-based.

- The *application logic (A)*, or business logic layer, is responsible for the functions to be made available to the user.
- The *data access logic (D)* layer handles information management, possibly accessing databases or legacy systems (a *legacy system* refers to a system inherited from the past and developed with older generation technologies, such as a COBOL application, which is still very important for the organization), or even information resources available on the internet.

The presentation layer constitutes the *front end* of the application, i.e., the part of the application component dedicated to user interaction. The application logic and data access layers constitute the *back end* of the application and are accessible through methods based on APIs (Application Programming Interface). Unlike GUIs, which are designed for human users, APIs are aimed at other software. In this way, different applications can be integrated. Drawing a comparison with typical programming languages (e.g., the C language), an API for application logic appears as a list of method signatures[1]. Regarding the data access layer, APIs can also take the form of methods that receive an SQL query as input and return the result of that query.

From an architectural point of view, logical layers can be contained within a single *software artefact* in the case of monolithic applications, or split across different *software artefacts* that communicate with each other via appropriate interfaces (Fig. 6.1) in the case of distributed applications. In the latter case, one can distinguish between *closed systems*, in which each logical layer is designed to communicate only with adjacent layers, and *open systems*, where each logical layer can also communicate with external systems (e.g., other logical layers from other applications).

Applications designed according to the distributed model are more flexible and scalable, and for this reason, they are the most widely used in current practice.

6.2 Functional Components

An information system is a complex system composed of several applications, which in most cases are *modular*: a module is defined as an autonomous software component associated with one or more functionalities, and therefore with specific functional and application objectives. The architecture is thus partitioned into functional modules, each of which plays a specific role. Often, the partitioning into modules reflects the functional division of various activities in the company (consider, for example, the functions identified by Porter's value chain at the operational level).

[1] A method's signature consists of the method's identifier and the list of input parameter types it accepts

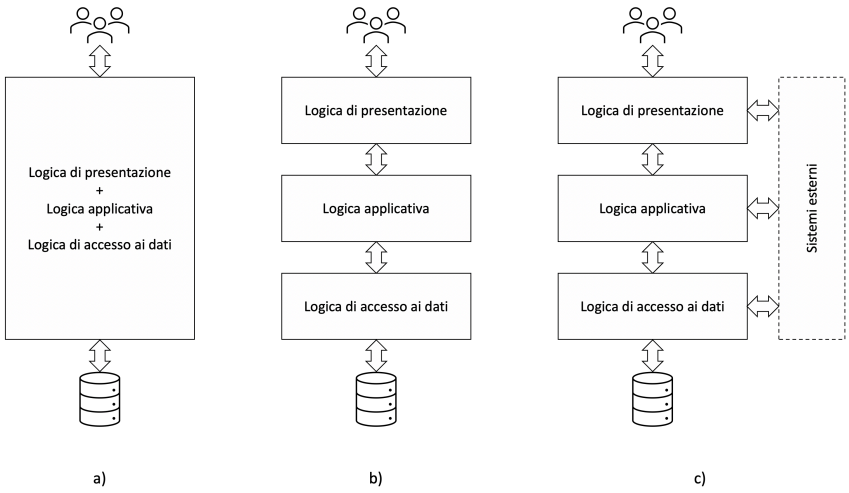


Figure 6.1: a) Monolithic applications b) Closed distributed applications c) Open distributed applications

Modularity is a fundamental characteristic because it improves the maintainability of the architecture. Especially in open systems—i.e., those structured to cooperate with other systems over a network—where interconnection with other firms fosters change and organizational dynamism, internal architectures are often subject to change. Modularity allows a module to be replaced or its functionality changed without affecting the entire architecture, but only the impacted module(s). In such cases, the change might not even be visible at the interface level, and therefore does not alter other modules or their cooperation. Furthermore, information systems in organizations are so complex that building them from scratch is nearly impossible or, at the very least, not economically viable. For this reason, it is usually better to opt for already available modules on the market (i.e., *COTS* components), and only develop specific functionalities. With this approach, when facing the classic Make-or-Buy decision, the Buy option is increasingly favored—at least for standard functionalities—over the Make option.

Two of the applications that companies most frequently acquire using the "Buy" approach are *ERP (Enterprise Resource Planning)* and *CRM (Customer Relationship Management)*, which are described in the following sections. According to what is described in Chapter 5.1, ERP and CRM are application-level technological components.

6.3 ERP

ERP (Enterprise Resource Planning) is a software suite that offers modules to support the operational system and thus the internal transactions within the organization aimed at enterprise management. An ERP system provides functionalities

that fully cover Porter's value chain.

6.3.1 History of ERP

To understand the emergence of ERP systems, one must trace back to the advent of CIM (Computer Integrated Manufacturing) systems, which became widespread in organizations during the 1980s and focused on production activities in manufacturing companies. In fact, starting from activity monitoring, data integration and evaluation from individual machines, and data related to the organization of cells, production areas, and plants, CIM aimed to provide an integrated view of the production system, supporting its various sectors (design, engineering, production, quality control, production planning, and marketing). This was mainly to support decisions related to process efficiency optimization.

Alongside CIM systems, MRP (Materials Requirements Planning) systems also emerged in the 1980s, with the following key functionalities: (i) scheduling material requirements by automating the bill of materials and, based on production targets (e.g., number of products to be produced), automatically defining the list of raw materials to purchase, (ii) scheduling operations, and (iii) inventory control. MRP systems were soon absorbed by MRP II (Manufacturing Resource Planning) systems. These are modular systems designed to replace any other sector-specific system for managing raw materials and operations. Indeed, MRP II systems deal with planning and managing all business resources and processes at the operational level: materials, production, personnel, maintenance, financial and administrative resources.

ERP emerged in the 1990s to further expand MRP II systems. The first version of ERP directly connected accounting management with logistics management (warehousing and procurement). Subsequently, internal relationships were also implemented with other operational level areas (e.g., sales, distribution, production, plant maintenance, project management, etc.).

Since the early 2000s, major ERP solution providers have begun creating vertical IT solutions for various business sectors; thus, specialized applications began to emerge for the automotive sector, retail, logistics, mechanical industry, and construction industry.

In recent years, the ERP domain has also seen widespread adoption of technologies that enable data analysis to extract knowledge and build models, including predictive ones, thanks to Artificial Intelligence and Machine Learning techniques. This chapter primarily examines aspects related to supporting operational activities, while data analysis technologies are treated separately in Chap. 7. These features are currently an important and constantly evolving part of ERP systems. Currently, evolution also focuses on new functional aspects supporting sustainability in organizations, for example, generating reports on a company's environmental impact.

6.3.2 Properties of ERPs

In addition to the functionalities provided by early IT systems, it should be noted that until the early 1990s, information systems in companies followed a design and development cycle known as the *silo approach*: applications were autonomous islands supporting a well-defined business function. These islands contained all software components (database, application logic, and presentation) necessary to perform their intended tasks. This approach worked well as long as the applications did not need to interact. The need to integrate or simply communicate applications highlighted the weaknesses of this approach, such as: (i) the need for significant IT effort to manage information flows between applications, (ii) data redundancy, which caused misalignment and fostered errors and inconsistencies, and (iii) difficulty in obtaining an overall view of data across different subsystems.

These problems encouraged the spread of ERPs, which are characterized by three fundamental properties:

- Uniqueness of information.
- Modularity.
- Prescriptiveness.

Uniqueness of information refers to a single logical representation of data valid across all ERP modules. All modules see the same values: this resolves redundancy and data inconsistency issues.

Regarding *modularity*, the suite is composed of self-sufficient modules that can be acquired and integrated incrementally. Each module consists of functions that generally support elementary activities (e.g., material reception). In addition to benefits in terms of application flexibility and scalability, modularity enables new COTS acquisition strategies. Indeed, one can purchase modules from a single provider (*One Stop Shopping Strategy*) or from different providers (*Best of Breed Strategy*). In the former case, interoperability between modules is guaranteed. In the latter, interoperability is not guaranteed, but there is an opportunity to acquire modules from highly specialized providers, hence of better quality, to perform various functions. Clearly, in the latter case, significant integration effort is required.

The third characteristic of ERPs, *prescriptiveness*, is linked to the fact that these products incorporate the company's "operating" logic, also codified in business rules. For example, production and logistics processes can be regulated by prescribing that a material cannot enter without an order or that a material can be ordered if and only if it has been requested by an authorized business unit. In this way, it is the company that must adapt to the software and not the other way around. This could be an advantage for companies adopting ERP, as the embedded logic is usually optimal, being designed based on best practices or well-known and optimized processes.

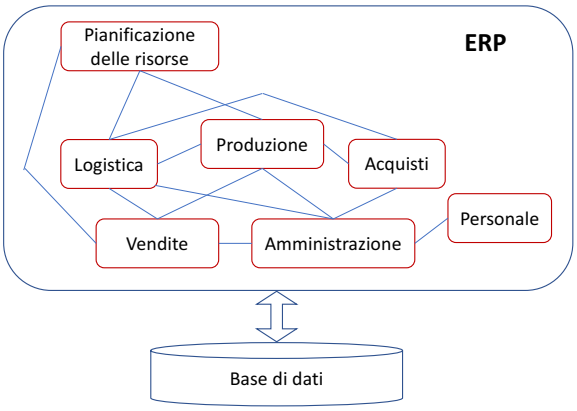


Figure 6.2: Organization of an ERP system (adapted from [19])

6.3.3 ERP Functionalities

Figure 6.2 shows a schematic representation of the organization of an ERP system. The modules represented are some of the main functionalities implemented according to best practices. These modules can be classified into three categories: institutional systems, sector-specific modules (primary activities), and managerial modules (see Figure 6.3). Institutional and managerial modules are horizontal, meaning cross-sectoral. Sector-specific modules are vertical, meaning they are specific to a given business context. Managerial modules provide data processing aimed at offering an overview of the activities carried out at the operational level, including elements such as dashboards, reporting, and advanced economic or business intelligence analysis.

Figure 6.3 represents a so-called *core* ERP, i.e., the main system. Subsequently, the core ERP has been extended to meet new needs, especially in terms of interaction with customers and suppliers. New applications have been developed, giving rise to the *extended* ERP, as shown in Figure 6.4. PLM (Product Lifecycle Management) is an application capable of managing product information throughout its entire lifecycle—from product conception to realization. SCM (Supply Chain Management) is a software for managing the supply chain that provides information to support purchasing decisions and production scheduling. In particular, this information helps companies plan their supply chain (supply chain planning) and execute the steps of the supply chain (supply chain execution). CRM (Customer Relationship Management) supports customer interaction, from initial contact to after-sales, with the goal of providing advanced services to foster customer loyalty and increase satisfaction.

Note that these extension modules may be part of the ERP suite as extension modules, but alternatively, they can also be acquired separately from other vendors and therefore appear in the architecture as external modules with which the ERP system must interact and be integrated.

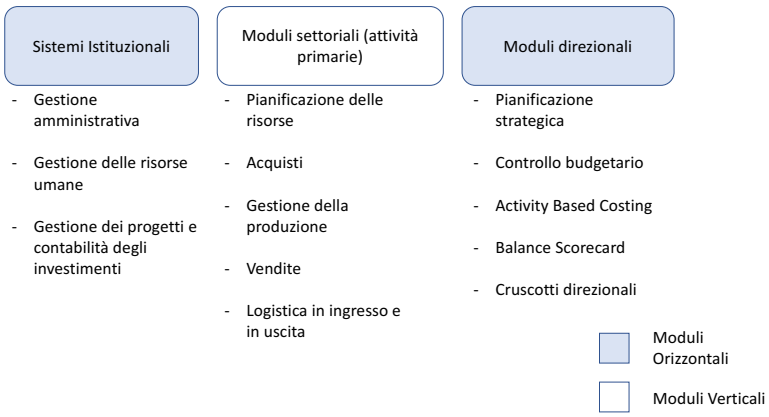


Figure 6.3: ERP Suite

6.3.4 Architectural Aspects

From an architectural point of view, three main approaches can be identified, shaped by the evolution of technology over time (see Figure 6.5). In the 1990s, ERP was installed in organizations using the client-server paradigm. In this setup, all functional modules were installed on one or more servers, and users accessed the functionalities via clients. With the spread of the Web, ERP vendors began offering *Web-based ERP*: ERPs developed as web applications and thus accessible through a browser. This shift led from two-tier architectures to three-tier or more. This innovation enabled companies to access their ERP systems remotely, offering users the ability to use ERP functionalities even outside the organization. For data analysis, modern ERPs have adopted OLAP-based technologies for the analytical part, allowing efficient, complex queries on large volumes of data. They have also developed analytical modules using data mining and machine learning techniques. Another major ERP innovation coincided with the spread of Cloud Computing: many ERP vendors now offer modules in SaaS (Software as a Service) mode. These architectural details will be described in more depth in the following chapters.

In any case, the architectural evolution of ERP has led to decreased costs for acquisition and management. Web-based ERPs allowed outsourcing of IT system management, significantly reducing costs. These costs were further lowered by SaaS ERPs because (i) service-oriented development logic reduces vendor costs and thus sale prices, and (ii) pricing is consumption-based and typically results in lower total costs than traditional acquisition (license and implementation costs).

6.3.5 ERP Implementation

As mentioned earlier, ERPs are COTS applications. As such, they are not designed and built for a single company but for a set of companies in a specific industry (or

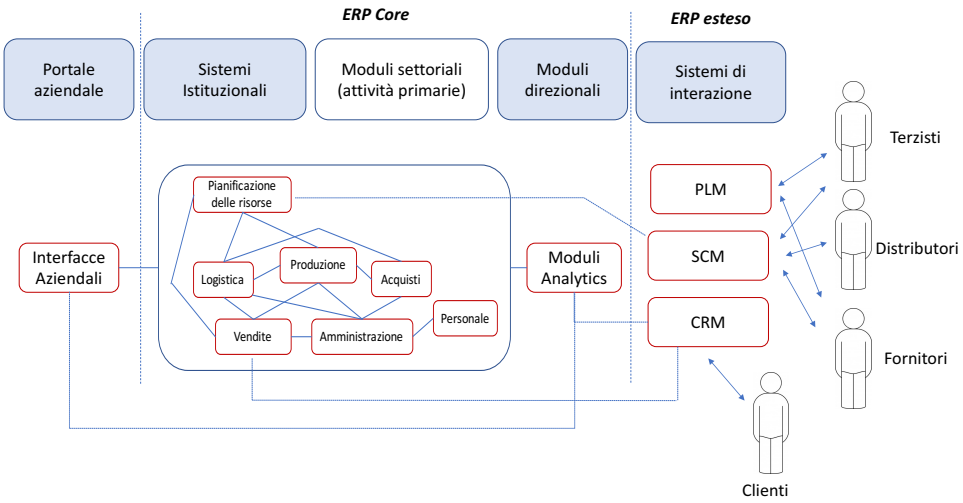


Figure 6.4: Extended ERP

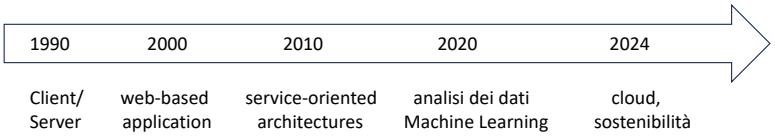


Figure 6.5: ERP Evolution

application domain) or sharing some characteristics. Therefore, different suites are available for different sectors. Suites designed for manufacturing companies offer modules reflecting Porter's value chain functionalities. However, for example, chemical and steel industries follow a slightly different model, and the developed software suite will thus comprise different modules. Figure 6.6 shows a possible segmentation of ERP modules for such companies. Note that Supply Chain Management is not included in ERP as it is usually an extension.

Figure 6.7 illustrates instead a software suite for telecommunications and energy service companies. An ERP supports almost all operational activities; only network planning and design are excluded.

In any case, even though ERPs are verticalized for a specific sector, companies within the sector often have different needs. Therefore, once installed, they are not immediately usable but must be properly configured to meet business requirements as much as possible (see implementation process in Figure 6.8). The first step in this direction is parameterization: by setting certain parameters, functionalities can be enabled or disabled, or a specific version of a function can be selected. This builds an application instance. If the company wants to make further changes to the product, it must proceed with actual customization, which involves changing

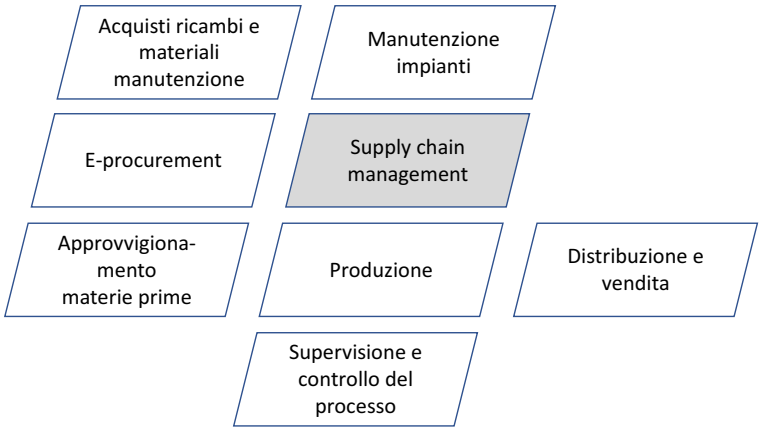


Figure 6.6: Main modules offered for chemical and steel companies (adapted from [6])

the code. This phase can be likened to a software project aimed at modifying an existing program.

6.3.6 ERP Benefits

Adopting an ERP affects not only management processes but also how the company positions itself in the market. In other words, this intervention brings benefits in terms of both efficiency and effectiveness. Efficiency benefits relate to time and cost savings in operational activities obtained through procedure automation and integration. Effectiveness benefits are linked to data uniqueness, the ability to process and transmit information, and IT platform standardization: there is, in fact, the possibility of greater data availability, accessibility, and timeliness. This enables access to the data needed to support tactical and strategic decision-making.

6.4 CRM

CRM (Customer Relationship Management) is a software suite that supports organizations in interactions with customers. CRM places the customer at the center of the business strategy. CRM is typically used to better understand customer needs and behaviors and to improve the use of customer interaction channels. In particular, the goal of CRM is to retain customers by building long-term personalized relationships capable of (i) increasing customer satisfaction and (ii) increasing the value of the company for the customer and of the customer for the company.

CRM is often seen as an extension of ERP because it provides additional functionality for customer management, from the initial contact to after-sales service. Unlike ERP, however, only certain companies can benefit from its implementation.

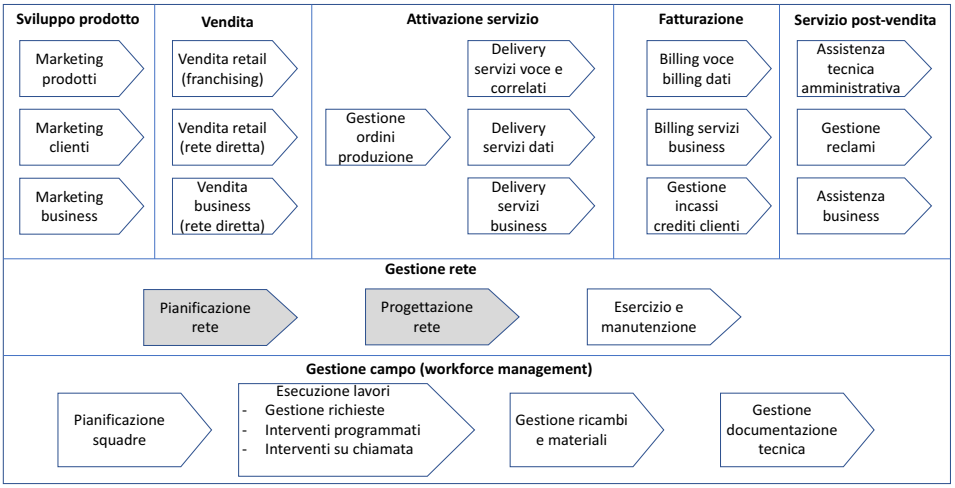


Figure 6.7: Main modules offered for telecommunication and energy service companies (adapted from [6])

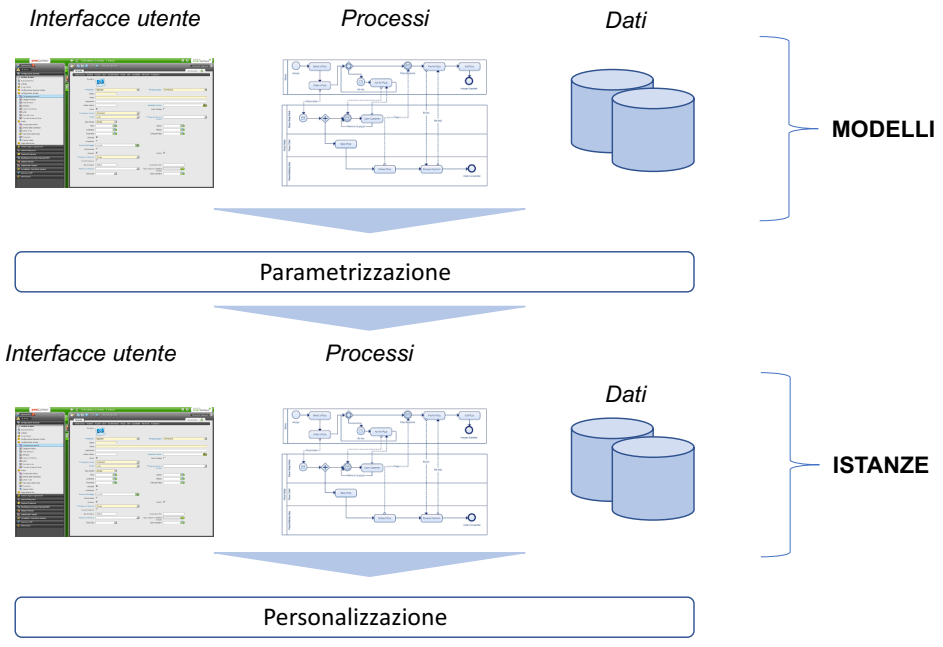


Figure 6.8: ERP Implementation

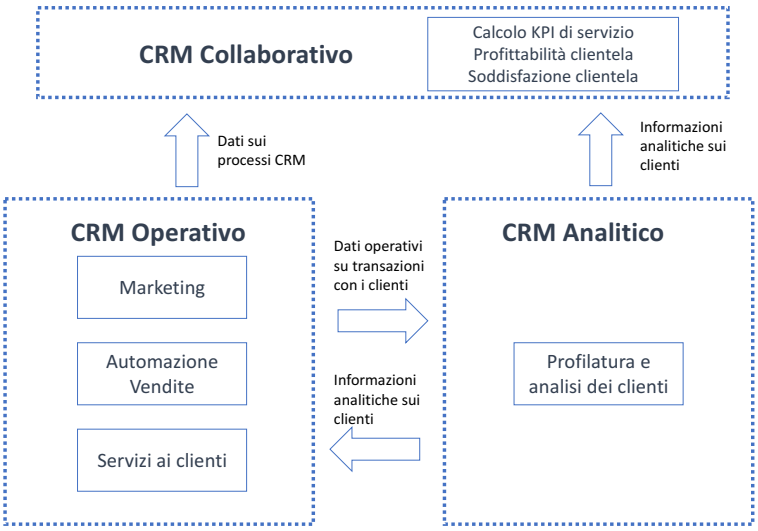


Figure 6.9: Components of an Enterprise CRM

In fact, the opportunities created by CRM are greater in sectors where there is higher relationship intensity with customers (frequency of interaction, duration of the contractual relationship), a sizable customer base, the potential to establish customer loyalty, and multichannel interaction. The latter refers to the fact that a customer can access an organization's services through various channels (e.g., physical locations, web, phone). In short, CRM systems provide more functions than the commercial logistics systems in ERP suites. Key aspects of CRM include multichannel customer relationship support (inbound and outbound contact), post-sales support (service) and loyalty management, and analysis of customer behavior.

It is important to understand that CRM is not just an application, but a true strategy that the organization must pursue at a company-wide level. Although CRM consists of different software modules and technologies, it should be considered a system supporting a process aimed at improving business objectives. Implementing a CRM helps better characterize customers and design personalized marketing campaigns with the goal of increasing returns.

The two main components of CRM are operational CRM and analytical CRM. *Operational CRM* is the front-end module that supports daily customer interactions, while *analytical CRM* is considered a back-end module that focuses mainly on customer data analysis. Figure 6.9 shows the components of CRM and the communications between modules. The figure also shows a third module, *collaborative CRM*, which is responsible for calculating indicators to be shared with various stakeholders within the company (e.g., sales and marketing managers, head of sales agents).

Figure 6.10 expands on the previous figure by focusing only on the interaction between operational and analytical CRM, highlighting especially the components

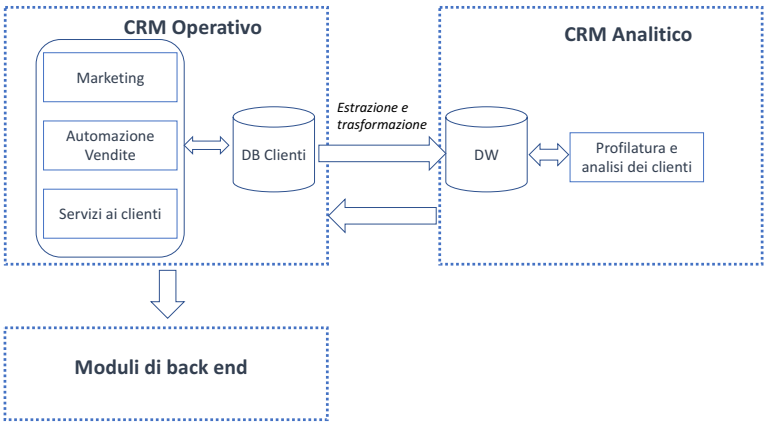


Figure 6.10: General architecture for data management in a CRM

Marketing	Vendite	Servizi ai clienti
Generazione liste clienti	Gestione vendite	Contact center
Gestione campagne	Gestione contatti	Web based self service
Cross-selling e Upselling	Gestione opportunità	Call scripting

Figure 6.11: Components of Operational CRM (adapted from [4])

needed for data exchange and management. Operational CRM records all customer interactions with the company across various channels into a single database. Analytical CRM relies on a data warehouse built from the information extracted from the operational CRM database. The functioning of the data warehouse will be explained in Section 7.1. The figure also shows that operational CRM data also feeds into back-end systems. In fact, customer interactions often generate events that must be handled by back-end systems—for example, a sales request must then be managed by an ERP.

In the following sections, we describe the composition of CRM modules as presented in Figure 6.9.

6.4.1 Operational CRM

Operational CRM deals with customer interaction with the company and, as described in Figure 6.9, it consists of various components to support sales, marketing, and customer service activities. These components rely on other modules, as described in Figure 6.11.

Marketing Company strategy no longer focuses solely on expanding the customer base and selling products to as many customers as possible, but rather on maximizing loyalty among existing customers and thus selling more products to

already acquired clients. This is achievable only through well-designed marketing campaigns targeted to individual clients or small client groups. It should be emphasized that the success of a marketing campaign is usually proportional to the company's ability to acquire and analyze data.

Systems that a marketing department may implement to increase customer satisfaction include:

- *Customer list generation*: produces lists of customers suitable to receive marketing communications. These lists are compiled through customer segmentation based on data related to interactions with the company (e.g., website visits, surveys, call center contacts). The segmentation is provided by analytical CRM.

- *Campaign management*: this module, also known as Marketing Automation, refers to tools designed to automate marketing tasks and processes. Specifically, it guides users in defining, planning, scheduling, and analyzing marketing campaign results. The benefits a company can gain from implementing this module are significant: (i) improved efficiency and reduced errors, (ii) increased contact conversion rates (the percentage of contacts turned into customers), (iii) enhanced marketing productivity and effectiveness, (iv) strengthened customer relationships.

- *Cross-selling and Upselling*: Cross-selling refers to selling more products or services to a customer, while upselling refers to strategies aimed at increasing the value of sales to a customer. For example, a restaurant performs cross-selling by offering an extra dish, and upselling by suggesting a more expensive dish than the one originally chosen. This module helps the marketing department implement these strategies by providing all relevant customer data and analyses.

Sales Automation The sales department was the first to adopt CRM, with one of the earliest components developed: *Sales Force Automation (SFA)*. SFA is a module used by sales agents to support them throughout the sales process: customer identification, contact management, offer definition, and purchase recording. Sales support modules now manage all channels and are divided into:

- *Sales management*: the sales management module automates or supports all the phases of a sales process illustrated in Figure 6.12. In particular, the module handles contact selection, assists the sales agent in drafting the offer and contract, and, by recording all operations performed by agents, enables performance analysis. The web channel provides all functionalities for online purchases: catalog management, shopping cart management, order finalization, and payment.

- *Contact management*: handles customer data and identifies potential customers for future sales. The contact management module also takes into account additional information that can help improve customer relationships.

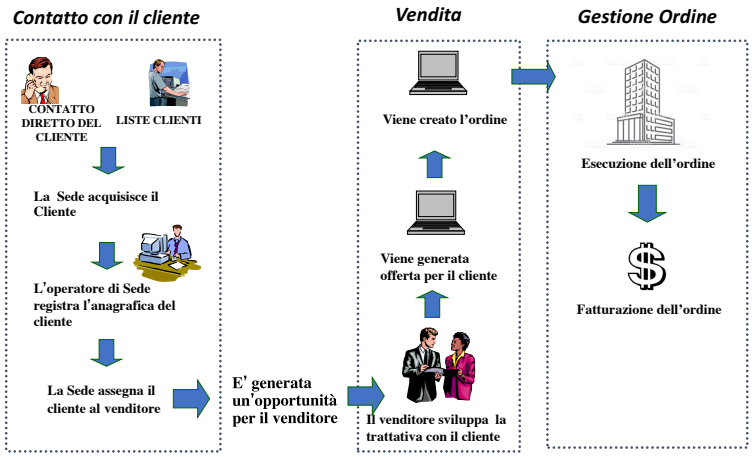


Figure 6.12: Main steps of a sales process

For example, equipping a call center with functions to identify the caller and retrieve notes on previous interactions can allow greeting the customer by name and personalizing the conversation based on past communications—this can make the customer feel valued and further build loyalty.

- *Opportunity management*: this module aims to find new customers or organizations for future sales. Its functionalities are also designed to calculate the likelihood of a sale associated with a customer, saving time for sales agents and marketing on analyzing and contacting unprofitable leads. The main difference between the contact management and opportunity management modules is that the former deals with people and organizations who are already customers, while the latter focuses on new leads.

Customer Service Sales and marketing are the business units that interact the most with customers before a sale. For this reason, the support modules for these activities are considered among the most important in CRM. Unfortunately, many organizations overlook the importance of continuing to nurture customer relationships after the sale has been made. Maintaining relationships in the post-sale phase is often even more important for strengthening customer ties and building loyalty. For this reason, CRM also offers a Customer Service module, which consists of the following submodules:

- *Contact center*: contact centers (or call centers) are responsible for managing inbound calls (customers calling the company) and outbound calls (company calling customers). CRM records all customer interactions with the call center and provides features for caller identification and call handling. Common functionalities used by call centers include (i) automatic call distribution, (ii) IVR (Interactive Voice Response), where users provide or receive information

through phone keypad input, (iii) predictive auto-dialing, which automatically calls customers and selects the right operator to handle the call.

- *Web-based self-service*: allows customers to use the web to find solutions to their problems or to contact the company (e.g., forms or online chat).

- *Call scripting*: this module manages knowledge by providing operators access to a database containing solutions to common customer problems. The system can also provide a list of questions that the operator can ask to identify the root issue and appropriate solutions.

6.4.2 Analytical CRM

Operational CRM records all customer interactions in a unified database. Analytical CRM uses this data to analyze customer preferences and behaviors to extract meaningful patterns. This is enabled through data warehouses and data mining systems, described in Sections 7.1 and 7.2. These systems aggregate, analyze, and disseminate information quickly within the organization to support decision-making. Analytical CRM serves three main purposes: (i) *Reporting*, understanding who the customers are, their characteristics and preferences; (ii) *Analysis*, segmenting customers into categories (e.g., active, dormant, inactive); and (iii) *Predicting*, forecasting customer actions and desires. This helps identify new opportunities and deliver higher quality service (efficiency, responsiveness, attention to needs, etc.).

Analytical CRM sends analysis results to operational CRM. For example, it supports the Marketing module by providing customer segmentation and identifying potential campaign targets. It also supports Sales by recommending products/services based on customer preferences and purchase history.

6.4.3 Collaborative CRM

Collaborative CRM aims to compute metrics relevant to the entire company, such as customer profitability and satisfaction, and to share this data and insights from analytical CRM with internal stakeholders. These indicators help evaluate the CRM's impact (e.g., increased returns) and can be used to assess whether the investment met expected objectives.

6.5 Questions

- Describe the general characteristics of ERP systems and discuss their representation in an ArchiMate diagram.

- What is meant by an ERP system? Describe its operating principles, benefits, and general functional architecture.

- Explain the functional architecture of ERP systems and discuss the pros and cons of make vs. buy approaches.

- Explain the role of data in an ERP system.
- Describe the characteristics of CRM systems and their functional architecture.

Chapter 7

Data Analysis

7.1 Data Warehousing

Data are an important resource not only at the operational level, but also at the tactical and strategic levels to support decision-making processes within an organization. This creates the need to access aggregated data and perform complex queries, such as analyzing correlations between different variables (for example, the relationship between a customer's place of residence and their likelihood of using a certain category of services). As one moves from the operational level to higher levels of Anthony's pyramid, both the type of data of interest and the way they are used change. For tactical and strategic activities to be successful, it is necessary to provide tools capable of extracting relevant information from different sources (both internal and external) quickly and effectively. These tools fall within the application area called *Business Intelligence (BI)*.

The first decision support tools were reports. Reports contain analytical data, obtained from analyzing the operational database, produced in a predefined format at regular intervals. These tools suffer from several limitations, as the data they contain are static and the user cannot interact with them to obtain details or additional information on unforeseen aspects. Additionally, generating reports can be complex and time-consuming, making it impossible to access up-to-date information. Finally, reports provide a limited view of aspects relevant to strategic decisions because they are based only on the organization's internal data extracted and aggregated from the operational database.

A more advanced and in some ways more dynamic tool is the spreadsheet (e.g., Excel). In this case, analyses can be defined directly by the end user, who can modify and execute them as needed. Despite these advantages over reports, spreadsheets are still limiting due to the complex processes required to export data from the database and import them into the spreadsheet. Moreover, each user can create a personalized tool that, although flexible, lacks the benefit of integrity and correctness checks typical of tools shared among multiple users.

A tool capable of supporting activities at the higher levels of Anthony's pyramid

is a database with characteristics different from traditional operational databases:

- The database can include data from operational databases and external sources.
- The data are organized in a structure that facilitates analysis.
- The database structure highlights all and only the relevant data.
- Data sources are integrated and updated.
- Analysis tools are available with relatively short response times, considering the objectives of the analysis.

To meet these needs, the concept of a *data warehouse* [19] was introduced.

7.1.1 Data Characteristics

A data warehouse is an OLAP system (Online Analytical Processing, introduced earlier). This differentiates data warehouses from traditional DBMSs, which are based on OLTP (Online Transaction Processing) technologies with many short online transactions (using INSERT, UPDATE, DELETE operations).

Analytical data used to support strategic and decision-making activities, as well as some planning and control activities, differ from operational data in several ways:

- *Objective*: data are used to make decisions about the organization's future developments and to identify organizational issues or growth opportunities.
- *Users*: data users are high- and mid-level managers who use information to improve organizational performance.
- *Time Horizon*: while operational data mainly reflect the present, analytical data incorporate historical data related to current data to identify issues, trends, and periodic patterns.
- *Level of Detail*: analytical data are usually aggregated from operational data or other sources at different levels of granularity.
- *Access*: unlike users of operational data, analytical data users access the information in read-only mode.

Given these characteristics, the classic relational model is not sufficient for building an analytical database.

To represent data in an OLAP system, the *multi-dimensional model* is used. In this model, information is represented through the concept of a *hypercube*, formed by a variable number of *dimensions* N, where each dimension is an analysis axis for the database. Each element is a database object, that is, a record whose value is defined by the coordinates represented by the analysis dimensions. The components of a multidimensional database are:

- *Fact*: a hypercube element obtained by specifying a value for each dimension.

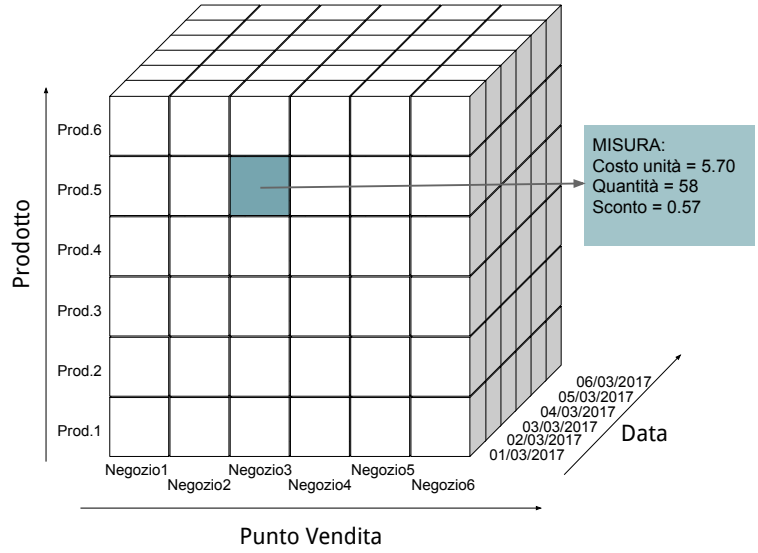


Figure 7.1: Three-dimensional hypercube representing key aspects for sales analysis in a supermarket chain

- *Dimension*: coordinates of each element corresponding to analysis dimensions.
- *Measure*: quantitative value of the elementary fact.

An example of a three-dimensional hypercube for analyzing sales in a supermarket chain is shown in Figure 7.1, where the analysis dimensions are time, product, and store location, while the fact is the sale and its measures include quantity sold, unit cost, and discount applied. Each dimension represents one of the coordinates of the fact and has a discrete value domain that defines the set of relevant facts. When a dimension's values are not discrete, they must be discretized, for example, by applying thresholds to define value intervals. Dimensions can be numerous and organized into hierarchies, where the level of granularity varies from level to level. Dimension hierarchies are organized such that a functional dependency exists between a higher granularity level and a lower one, such that a $1...N$ relationship exists between them. In the example of Figure 7.2, products are divided into several categories. Each category contains one or more product types, and each type includes multiple specific products. Along a hierarchical line, relationships define different aggregations for the dimension considered. Returning to the hypercube in Figure 7.1, the product dimension can be manipulated by changing the level of granularity—for instance, considering product groups by category rather than individual products. In this case, the fact content will aggregate information for all products in the selected category. Similar operations on other dimensions, their granularities, or values allow selection of the desired level of detail for analysis.



Figure 7.2: Example of a hierarchy for the product dimension. Granularity increases from right to left.

7.1.2 Data Warehouse

The data warehouse is a database that collects summarized, integrated, structured, and historical data of interest for an organization. More broadly, data warehousing refers to the entire set of operations involved in populating, maintaining the database, and querying it to extract relevant information from the decision-support database. As mentioned, the data warehouse is an OLAP (On-Line Analytical Processing) system. As described in Chapter 1, the OLAP paradigm is characterized by a few transactions and complex queries requiring aggregation of large data volumes. OLAP systems are evaluated by response time efficiency and effectiveness. An OLAP system stores aggregated data, historical data, and uses multi-dimensional schemas (typically star schemas). Normally, queries access large amounts of data to answer complex questions, such as the net profit achieved by a company in a given geographic area last year.

OLAP systems support complex and ad hoc operations, where each operation may involve many aggregated, historical, and often non-current data. For this reason, the ACID (Atomicity, Consistency, Isolation, Durability) properties are not critical, since operations are read-only.

Instead, OLAP systems adhere to the FASMI properties (Fast, Analytical, Shared, Multidimensional, Informational). Thus, the system must offer a multidimensional view of data (*multidimensional*) containing all relevant information (*informational*), enabling complex analyses (*analytical*) by multiple users with varying access rights (*shared*) and returning answers quickly (*fast*).

Moreover, a data warehouse must have the following features:

- **Subject-oriented**: focuses on key analytical entities. For example, sales, orders, and products may be analytical entities.
- **Integrated**: data are collected from both internal and external sources. Starting from heterogeneous data, they must be made consistent and integrated to provide a complete and clear view of the analyzed entities.
- **Time-variant**: the data warehouse stores historical data. To analyze event timelines, each data point is tagged with a timestamp indicating its reference period.

- **Non-volatile**: once stored in the data warehouse, data are typically not modified. Data are archived in read-only mode.

7.1.3 Data Warehouse Architecture

Data warehousing is the set of activities that lead to the definition, construction, and maintenance of information in the data warehouse, which can be defined as the database that collects all data of interest to the company, synthesizing, integrating, and structuring them. Within the data warehouse, it is possible to identify several databases organized hierarchically. At the first level, we have the *sources*, i.e., the origins from which the data populating the data warehouse must be extracted. These sources certainly include the organization's operational database, but also other databases or external sources that may be structured in various forms (databases, reports, big data, etc.). All these sources undergo a set of operations called *ETL* (Extraction, Transformation, and Loading), which allow for extracting the relevant data from the sources, transforming them according to the desired multidimensional structure, and loading them into the data warehouse. This population phase may use an intermediate database between the sources and the actual data warehouse, called the *staging area*. The data warehouse is the multidimensional database containing all the information of interest accessible by specifying the analysis dimensions. Depending on the presence or absence of the staging area, the DW architecture can be two-level (without staging area) or three-level (with staging area). Downstream of the data warehouse, there can be another type of database called a *data mart*. Data marts are small thematic data warehouses containing an extract of the information in the complete data warehouse. The latter can indeed have very large dimensions that are unlikely to be of interest to all system users. Using data marts, it is possible to define views of the data warehouse that contain a subset of the information. This subset may involve a reduction of the analysis dimensions (only those relevant to the target user category of the data mart are considered), the granularity level of the dimensions, or the selected time horizon. An architecture that uses only the data warehouse is a centralized architecture in which all data is contained in the sole centralized multidimensional database. On the other hand, using data marts enables a distributed architecture in which the different thematic databases can be located on different nodes.

As an example, we can refer to a supermarket chain that uses data warehousing to improve its operations. In the example, the information sources can be various. First, there are operational databases, in this case, the sales data and inventory databases. Also relevant for analysis could be information about advertising campaigns conducted by the chain. Other sources that can influence decisions may come from outside, such as weather data (sales of certain products may be influenced by weather conditions), and data from agencies specialized in sales analysis, such as Nielsen. This information is transformed and restructured using ETL transformations and then stored in the corporate data warehouse. Decisions can be made based on the interaction with the data warehouse, influenced by all the analyzed sources—for example, supply planning. Other areas, such as customer management or logistics planning, use a subset of the data in the data warehouse, which is why

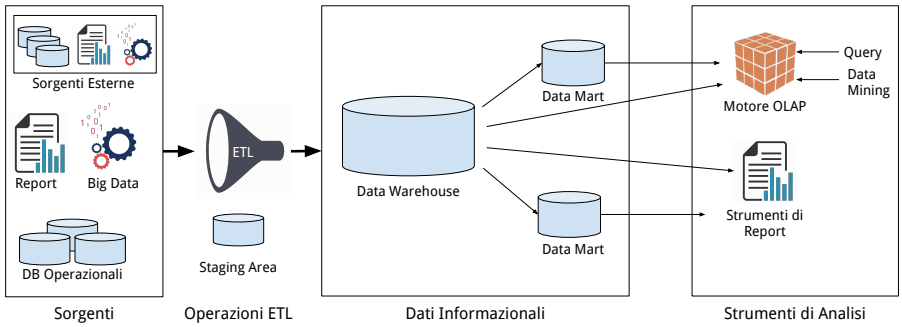


Figure 7.3: Three-level data warehouse architecture

thematic data marts can be created, queried through an OLAP engine, and used to extract automated reports.

ETL

Particular attention should be paid to the tools supporting the ETL process. These, as mentioned above, allow for the extraction, transformation, and loading of data into the data warehouse. As for *extraction*, it defines which data must be extracted and how they should be handled (e.g., whether to aggregate them at the source or extract them at the highest level of detail). Additionally, extraction can be: (a) *Static* if all data present in the sources are considered or (b) *Incremental* if only the data produced or modified since the last update of the data warehouse are considered. Following extraction, data needs *transformation* since it originates from heterogeneous sources and may be represented in different formats, making integration difficult. The main operations performed are:

- *Data cleaning*: Data extracted from sources often contains errors that must be corrected before insertion into the data warehouse. This operation is usually performed first in the transformation phase, though it may also be done after or during. The main data quality issues addressed at this stage are missing data, invalid values, and inconsistency between fields that should be related or have equivalent meanings.

- *Reconciliation*: The integration of different data must be preceded by this operation, which aims to relate data referring to the same "object".

- *Format standardization*: Data must be made homogeneous through standardization operations. These operations refer to (i) Joining or splitting fields: information present in multiple fields is combined into one, or information in one field is split into separate fields; (ii) Standardizing classification codes: different conventions for the same criterion are aligned to a single coding; (iii) Standardizing data formats: data represented in different formats are made consistent.

- *Duplicate detection and removal*: When duplicate information is present, this operation identifies and reduces it to a single instance.

Loading is the phase that transfers the data into the data warehouse according to the multidimensional model introduced in Section 7.1.1.

Metadata

The functions performed by ETL tools are supported and documented by metadata (data about data). Metadata is an important part of the architecture of a data warehouse and is usually stored in a repository that includes:

- *Data warehouse structure*: Metadata describes the structure of the data warehouse (schemas, views, dimensions, hierarchies, decomposition into data marts, and their locations).
- *Operational metadata*: Refers to data history and documents their origin and the transformations they underwent.
- *Metadata mapping operational data to data warehouse data*: Provides information on the sources and their content, transformation rules, and data warehouse update rules.
- *Usage statistics*: Metadata also describes how and how often the data warehouse is used.

7.1.4 Conceptual Model of the Data Warehouse

As mentioned earlier, data warehouses are based on the multidimensional data model, in which facts—elements of analysis—are represented through a set of measures, defined by dimensions that are the coordinates of the fact.

There are several conceptual models to represent multidimensional systems. The most widely used is the Dimensional Fact Model (DFM). In DFM, a fact is represented as a rectangle containing the corresponding measures. Dimensions are represented as labeled circles connected to the fact itself. Dimensions may be simple attributes of the fact or hierarchies, represented as trees, with the base dimensions as roots. Some hierarchy attributes may be optional, which is indicated in the DFM by a slash on the corresponding attribute line.

An example of a DFM representing the sales fact for the aforementioned supermarket chain is shown in Figure 7.4. The fact in this case is sales, with measures such as quantity sold, unit price, and any discounts. The fact is determined by four dimensions: the date the sale occurred, the location of the sale, the product sold, and the customer. Each dimension has an associated hierarchy, allowing for more or less detailed analysis of the facts stored in the data warehouse. For instance, it is possible to analyze the total quantity of sales for a specific product over the entire analysis period for stores in the city of Milan, grouped by customer gender. These insights result from aggregating across the various hierarchies of the

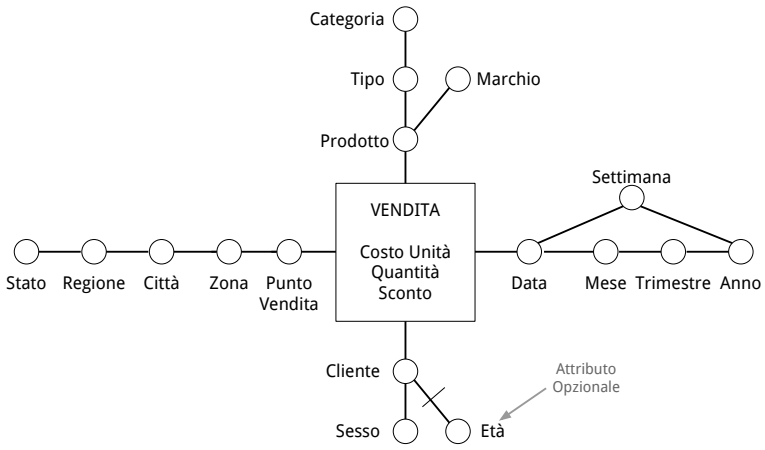


Figure 7.4: Dimensional Fact Model for supermarket sales

considered dimensions. In the modeled DFM, the age attribute for the customer is optional.

The DFM shown as an example corresponds to the hypercube relating to sales for the supermarket chain. However, the data warehouse will not consist of only one hypercube but of many, each modeling the dimensions related to an analytical fact of interest. For example, a company's data warehouse concerned with analyzing sales, customer support efficiency, and supplier interaction optimization will contain three different hypercubes—one for each analysis area: sales, customer service, and supplies. A DFM will be modeled for each of these hypercubes, defining the fact, measures, dimensions, and relevant hierarchies.

7.1.5 Logical Models of the Data Warehouse

Once the conceptual model of the multidimensional database has been defined, it must be implemented by translating the conceptual model into a logical model. It is therefore necessary to choose which DBMS to use to manage the multidimensional information.

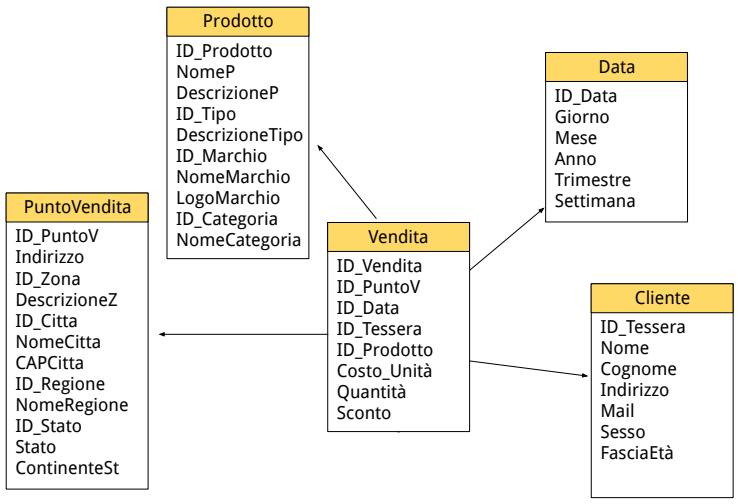
MOLAP Models The most direct choice might be to use a logical model of type MOLAP (Multidimensional OLAP). The MOLAP model translates the conceptual model into a multidimensional database that can be queried using multidimensional engines specifically designed to handle this type of data. This model has the advantage of exactly translating the conceptual model and of leveraging all the benefits of a multidimensional database, making queries efficient and fast. On the other hand, however, multidimensional databases are much less widespread than traditional relational databases and using them requires specific knowledge that the average user does not possess. Moreover, they are mostly proprietary models and languages

for which no widespread standard has been defined. Finally, the multidimensional model is based on the concept of a hypercube, in which a memory cell must be allocated for each possible combination of dimension values to store the corresponding fact, even if this fact did not actually occur and the cell remains empty.

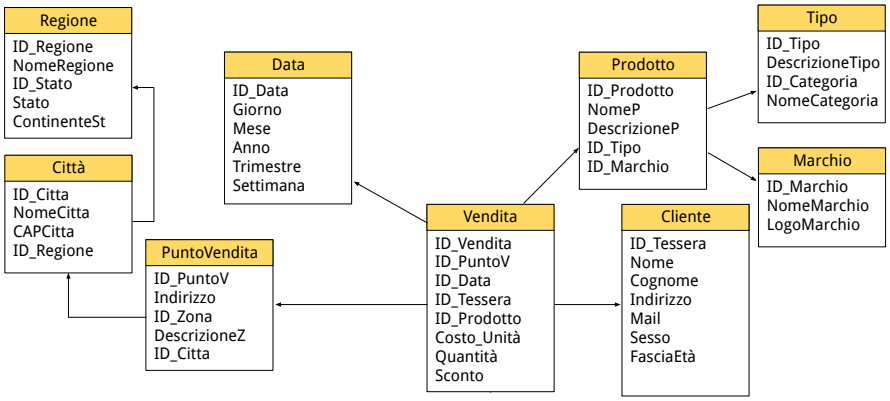
ROLAP Models To overcome these problems, an alternative to the MOLAP model is the use of a ROLAP (Relational OLAP) conceptual model. ROLAP models translate the multidimensional conceptual model into a relational model that represents the data using tables and allows queries to be performed using familiar SQL-based query languages, by performing JOIN operations on the tables representing the fact and its dimensions. The main advantages of this model stem from the familiarity with relational database querying tools, which allow users to interact with the data warehouse without specific knowledge. Furthermore, being based on the relational model, this representation requires less storage space than the MOLAP model, since each fact corresponds to a row in the database and it is not necessary to reserve space for facts that have not been recorded. However, this model has some disadvantages due to the difficulty in adhering to the data's conceptual model and the slowness and awkwardness of queries on a relational engine that is not well suited to handling multidimensional data.

HOLAP Models The HOLAP (Hybrid OLAP) model translates the multidimensional model in a hybrid way, usually using a relational database for the data warehouse, limiting space usage and allowing interaction with familiar tools. Multidimensional databases are instead used for thematic data marts, where the dimensions are inherently limited in number, but which gain the advantage of more efficient queries.

Multidimensional Schemas on Relational Databases In order to map a multidimensional database into a relational schema, it is necessary to identify the tables that will compose this schema. In the literature, two main approaches exist for this purpose: the *star schema* and the *snowflake schema*. In the star schema, two types of tables are used: the fact table and the dimension tables. The fact table contains all the attributes corresponding to the measures of the fact, and each row of this table thus corresponds to a specific fact. The dimension tables, on the other hand, contain for each dimension associated with the fact in the multidimensional schema, all the attributes related to the hierarchy corresponding to the dimension. A query will thus translate into a JOIN among all the involved tables based on the fact of interest. However, in this way, the concept of dimension hierarchy is lost, as it is flattened within a single table. An alternative approach is to use a snowflake schema, in which each dimension is associated with multiple related tables, allowing the preservation of some of the most relevant functional dependencies for users. A comparison between the two schemas is shown in Figure 7.5, where the star and snowflake schemas are presented in relation to the multidimensional model whose conceptual model is shown in Figure 7.4.



(a) Star Schema



(b) Snowflake Schema

Figure 7.5: Comparison between star schema and snowflake schema to represent the Sales fact in a supermarket chain

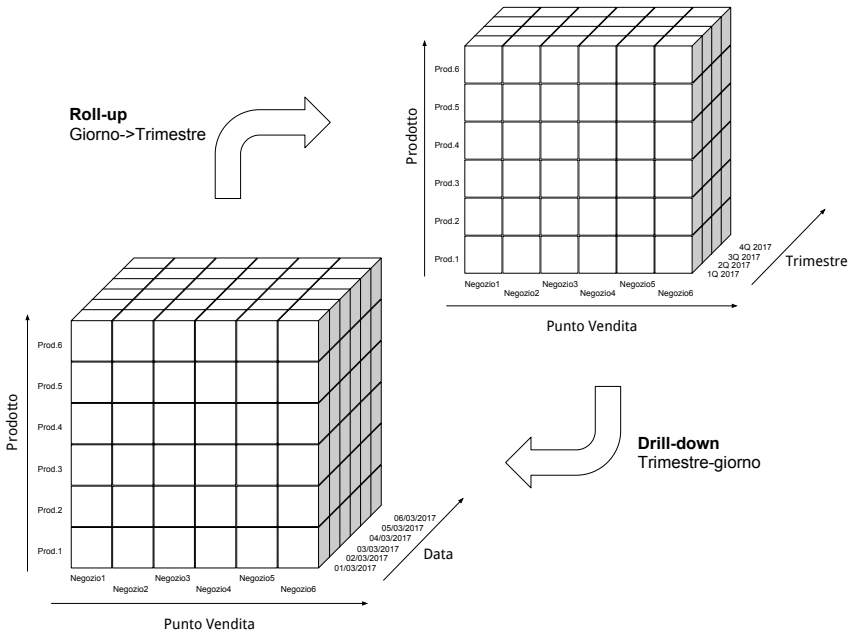


Figure 7.6: Example of roll-up and drill-down with aggregation/disaggregation on the *Date* dimension

7.1.6 Data Warehouse Operations

Data warehousing includes, in addition to a model of managerial data, a set of techniques that can be used to analyze the data. These techniques must allow even non-expert users to interact with the hypercube of facts to obtain more or less detailed information based on their needs, focusing on the dimensions of interest. The interaction usually begins with a hypothesis formulated by a user that leads to the extraction of the subset of data of interest, which can be explored further through successive interactions. Each interaction consists of applying an OLAP operator. The main OLAP operators are: drill-down, roll-up, slice, and dice.

The **drill-down** operation allows obtaining more detailed data by descending along a hierarchy of a dimension, thus moving from a higher level of aggregation to a lower one. Considering the DFM in Figure 7.4, a drill-down operation might go from a quarterly to a daily analysis of sales (see Figure 7.6). The opposite operation to drill-down is **roll-up**, where the level of detail is reduced by moving to a coarser level of granularity along an analysis dimension. An example could be moving from a monthly sales analysis to an annual one, in which monthly values are aggregated. Alternatively, a roll-up operation can be performed by eliminating an analysis dimension that is no longer relevant. For example, from a monthly sales analysis, one might remove the store dimension and aggregate based on the remaining dimensions (see Figure 7.7).

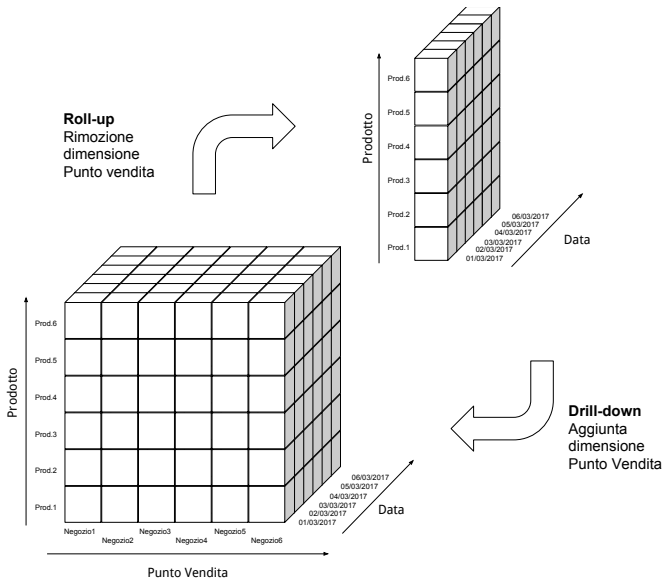


Figure 7.7: Example of roll-up and drill-down with elimination of *Store* dimension

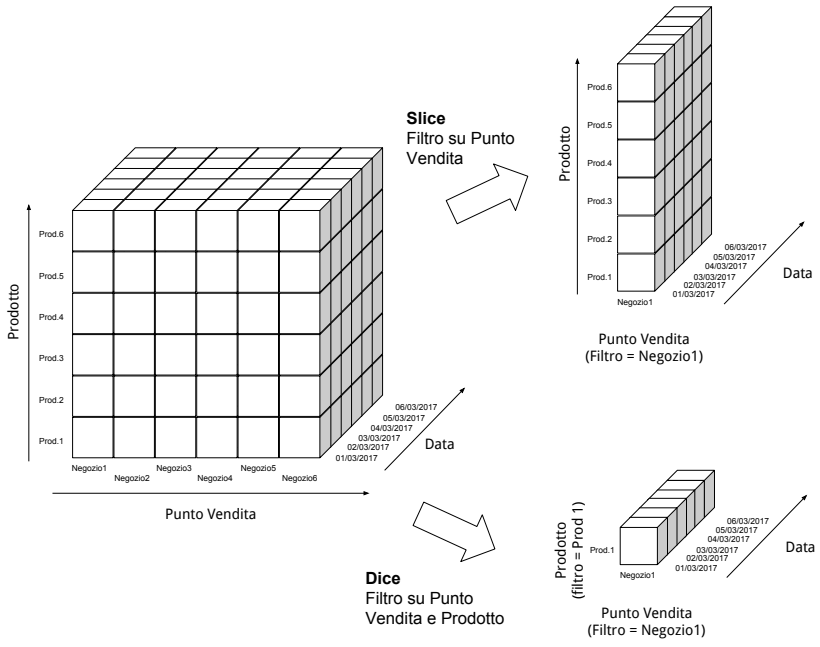


Figure 7.8: Example of slice and dice

The **slice** operator allows focusing the analysis on a portion of the data by fixing the value for one of the analysis dimensions. For example, one can focus on sales related to a single store, while maintaining all the details related to the other analysis dimensions. The result of this operation is a slice of the original hypercube, identified by the fixed dimension (see Figure 7.8). Note that, unlike roll-up, the slice selects a single store and does not aggregate over it. Another reduction operation is **dice**, which identifies a set of coordinates at any hierarchical level for any desired dimension. The result in this case is still a hypercube, but with a reduced set of data. For example, one could analyze sales data for a product (e.g., *product1*) in a specific store (e.g., *store1*).

7.1.7 Life Cycle of the Data Warehouse

As stated in Section 7.1.4, an organization's data warehouse will consist of multiple hypercubes, each containing relevant information for a fact of interest in the organization's managerial oversight.

The construction of the data warehouse thus follows an iterative approach. It begins by identifying the most relevant fact, then the first hypercube is modeled and populated. Subsequently, other hypercubes of interest are added and populated one by one. This approach has a dual advantage: on the one hand, it allows identifying relevant facts later, in response to new organizational needs; on the other hand, it limits the initial investment by developing the multidimensional database piece by piece.

The population phase of the data warehouse is a complex step that requires several operations. As already mentioned, ETL (Extraction, Transformation, and Loading) tools are used to populate the DW (see Section 7.1.3), where the final phase involves *loading* the data into the data warehouse. The data is inserted starting from the highest aggregation level and then descending to the more detailed levels.

During insertion, it is important to consider issues related to data updates, which may concern facts or dimensions. Updating a fact means inserting a new value for a combination of dimensions, or modifying an existing value. Updating a dimension, however, can lead to inconsistency. For example, if a customer's address dimension changes over time, one of the following decisions must be made: (1) update the address value for existing facts so that a customer is always associated with the same address; (2) retain the previous value without updating; (3) adopt a hybrid solution that tracks the different addresses over time and associates them with their respective time intervals. The last solution is the most complex to manage.

The data warehouse life cycle also highlights its limitations. It is important to note that the ETL process can take considerable time to execute. It is not necessarily an automatic task. Especially during the transformation phase, where discrepancies between data from different sources must be resolved, human intervention may be required. Therefore, not only is the creation of a data warehouse a costly operation, but its maintenance and continuous data updating also require significant effort.

This complexity results in an increased gap between updates in the source databases—which occur very frequently—and the moment when those updates be-

come visible in a data warehouse. To reduce this gap, in recent years, alternative approaches to data warehouses have been proposed. These often leverage big data algorithms and techniques that, although they do not require such a detailed design phase as in the case of the data warehouse, are capable of producing real-time analytical results.

7.2 Data Mining

Data warehousing supports the user's deductive processes by providing a set of tools for real-time interaction with the multidimensional database and for data navigation. This interaction allows the user to perform data analysis in search of specific information considered of interest by that particular user. This approach to directional data analysis within an organization can be insufficient, and further analyses aimed at automatically extracting hidden information inside the data warehouse can enrich the set of knowledge that can guide business strategies.

As defined in [19], *data mining* is the activity aimed at automatically recognizing and extracting hidden information from large databases, such as the data warehouse. The data mining process consists of several steps that can be iterated multiple times in order to refine the obtained information. This process, although largely automatic, requires user interaction in several phases. The user can indeed guide the iterative process by indicating the most interesting investigation directions. Moreover, even the final results offered by data mining analyses must be validated and selected by users, since part of the resulting information might prove to be of little interest.

In general, the data mining process consists of the following steps:

1. Selection of the data considered relevant for the analysis.
2. Data cleaning to eliminate inconsistencies and errors.
3. Integration of data coming from different sources to make them consistent.
4. Transformation and structuring of the data resulting from the previous phases.
5. Data mining analysis aimed at searching for noteworthy conditions within the analyzed data.
6. Evaluation of the results and their reduction to only those considered relevant.
7. Presentation of the final results to the user.

Steps 1 to 4 correspond to the steps necessary for the creation of the data warehouse, while steps 5 to 7 are typical of data mining alone. The architecture of data mining is shown in Figure 7.9.

Data mining functions can be classified into different categories [19, 31] briefly discussed in this paragraph. First, considering the possibility of training algorithms with ideal examples, data mining techniques can be distinguished into *supervised* and *unsupervised* learning techniques. *Supervised learning* techniques associate to

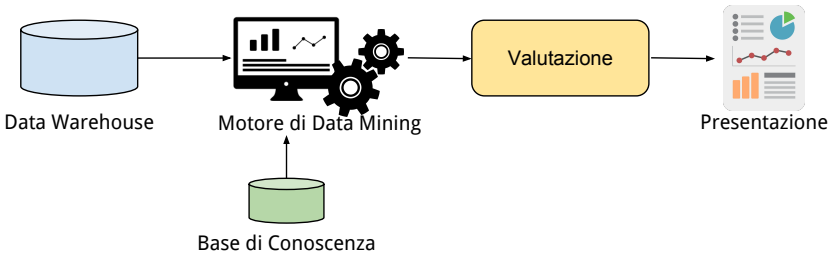


Figure 7.9: Architecture of Data Mining starting from the data warehouse

each considered record a target variable which can be, for example, a classification label of that data set. The operation of associating the record and the classification label is called "labelling." The target variable constitutes the desired output from the analysis of the record that is the input to the system. In *unsupervised learning* techniques, there is no target variable, and consequently, the goal is to identify meaningful recurring regularities or rules.

Another classification can be made considering the analysis objectives, distinguishing *predictive* and *descriptive* techniques. *Descriptive techniques* analyze past data and categorize, filter, and identify patterns by applying mathematical or statistical functions. *Predictive techniques*, instead, analyze past data to support predictions and decisions capable of anticipating events.

Considering the provided functionalities, we can identify several data mining techniques. Those described in this text are association rules, classification, and clustering. According to the categories described, association rules and clustering are two descriptive techniques with unsupervised learning, whereas classification is a predictive technique with supervised learning.

7.2.1 Characterization and Discrimination

Characterization and discrimination algorithms aim to extract relevant information that can precisely and concisely describe the data contained in the analyzed database.

In this category, for example, there is **generalization**, which starts from the principle that the number of elements a user can analyze is limited. The goal is thus to reduce the level of detail to reduce the number of elements to analyze. This reduction can be performed by generalizing the analyzed elements by finding common characteristics within the dataset. The reduction can be performed using aggregation techniques as those used in data warehouses (roll-up), or, if the attribute hierarchy is unknown, by removing attributes that have too high variability. For example, consider a case where a user wants to analyze sales data, considering revenue as the measure. The analysis dimensions could be address, city, and age group of customers, and day of sale. The address will be an attribute with high variability for which the dimensional hierarchy is unknown and can therefore be

removed. For the day of sale and for the city, instead, it is possible to roll up along the dimension until the attribute's variability remains below a certain threshold.

Other algorithms belonging to this category are **characterization** ones. In this case, the interest is to understand the nature of the data, thus studying their trends (maximum, minimum, mean, variance, etc.), the level of differentiation (for example, how many different values exist for a specific attribute of the database), and according to which distribution these values can be described. This type of analysis is often performed upstream to get an overview of the kind of data dealt with and their general characteristics.

7.2.2 Association Rules

Association rules describe the relationships occurring between different attributes of the database, aiming to identify conditions that frequently occur simultaneously. They are usually expressed with the formula $A \Rightarrow B$ where A is called the antecedent and B the consequent. Association rules can be used, for example, for market basket analysis, i.e., to identify products that a user often buys together. An example is the following rule:
$buys(X, cereals) \Rightarrow buys(X, milk)$

The validity of an association rule can be evaluated based on two values: support and confidence. *Confidence* measures the certainty of the found relation, i.e., how many times in the analyzed dataset, if the condition expressed in the antecedent occurs, the consequent also occurs. In probabilistic terms, this relation can be expressed as the conditional probability $P(B|A) = \frac{P(A,B)}{P(A)}$, that is, the ratio between the number of times A and B occur together (a purchase includes both milk and cereals) over the number of times A is observed (a purchase includes cereals). The *support* instead indicates the number of times the rule is verified over the total records in the dataset and is thus equal to $P(A, B)$.

As an example, consider analyzing sales data of products to find relationships between the sale of different products. Consider the following extract of the database:
$t_1 = \{p_1, p_2, p_3\}$
$t_2 = \{p_1, p_4\}$
$t_3 = \{p_1, p_3\}$
$t_4 = \{p_2, p_5, p_6\}$

where t_i represents a timestamp and p_i represents the sale of a product. If we wanted to extract the association rules that have both support S and confidence C greater than 0.5, we would obtain:
$p_1 \Rightarrow p_3$ with
$S = P(p_1, p_3) = \frac{2}{4} = 0.5$ and
$C = P(p_3|p_1) = \frac{P(p_1,p_3)}{P(p_1)} = \frac{\frac{2}{4}}{\frac{3}{4}} = 0.66$

$p_3 \Rightarrow p_1$ with
$S = P(p_1, p_3) = \frac{2}{4} = 0.5$ and
$C = P(p_1|p_3) = \frac{P(p_1,p_3)}{P(p_3)} = \frac{\frac{2}{4}}{\frac{2}{4}} = 1$

Both relations, being symmetrical, occur twice in t_1 and t_3 out of a total of four cases analyzed, so the support $P(A, B)$ in both cases will be 50%. Confidence instead depends also on the number of times the first term occurs in the analyzed cases. While p_1 occurs three times out of four (once independently of whether p_3 occurs), p_3 occurs only twice and always in conjunction with p_1. For this reason, the confidence of the two rules, calculated as $P(A, B)/P(A)$, is different.

Possible applications of association rules include planning effective promotional campaigns (suggesting products based on customers' previous purchases) and triggering preventive maintenance actions on equipment. An example of the first case can be seen on online shopping websites, where the display of a product is always accompanied by a list of other products frequently purchased together with the viewed product.

7.2.3 Classificazione

Classification allows automatically assigning elements of the database to predefined classes. Classification is a guided procedure, starting from an annotated set of elements, i.e., for which the class membership is known, and building a model able to identify the characteristics that allow assigning an element whose class is unknown to one of the analyzed classes. This model is then tested with a dataset for which the class is known to see if the classifier can correctly identify the category. The set of annotated elements used to build the model is usually called the *training set*, while the set of elements used to test the classification results is called the *test set*. The quality of the model is measured by its ability to correctly classify the elements composing the test set. Once a satisfactory model is obtained, it can be used on unclassified data to make a prediction on the class membership. Possible applications of classification include predicting the quality of suppliers or products, and predicting customers' propensity to purchase.

There are different types of classifiers based on various techniques including mathematical functions, Bayesian networks, decision trees, and neural networks. They differ according to a set of characteristics: (i) Accuracy — percentage of correctly classified elements; (ii) Speed — time to build the model and time for classification; (iii) Scalability — classifier's ability to build a model with an increasing number of variables; (iv) Robustness — classification ability in the presence of missing elements; (v) Interpretability, i.e., ease of interpreting the model results.

As an example of a classification model, consider *decision trees*, in which nodes are the attributes of the subject to classify, edges are the possible values of the attribute, and leaves are the classes to which elements can be assigned. An example of a decision tree is shown in Figure 7.10, where the goal is to predict whether a user will purchase an OLED television or not. The considered attributes are the customer's age (divided into age groups), their education level (university degree or not), and their residence (city center or suburb). The tree, built based on data relating to previous purchases of other users, can be used to decide whether to direct specific advertising for the product based on customer characteristics.

The construction of a decision tree proceeds by successive refinements. Starting

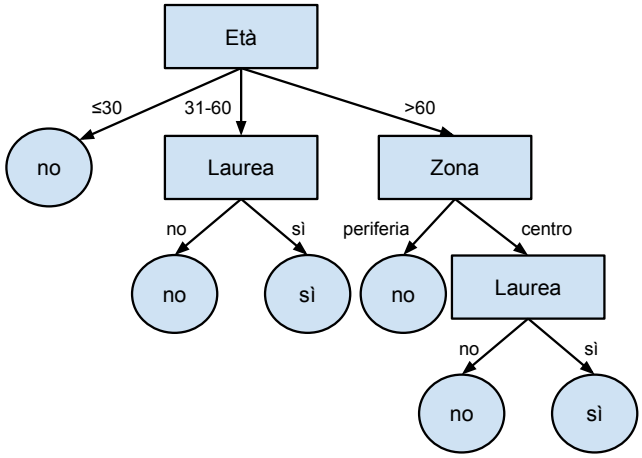


Figure 7.10: Decision tree for predicting the purchase of an OLED television

from a labeled training set, an attribute is selected as the root node (in the figure, age) and the dataset is split based on the different values assumed by that attribute (in this case discretized into three groups: less than or equal to 30 years, between 31 and 60 years, and over 60 years). The choice of the attribute is driven by evaluations related to *entropy*, which measures the amount of information extractable from the attribute, aiming to have a high entropy jump (for example, information gain) between tree levels and minimal entropy, ideally zero, for the tree leaves.

For each subclass, a further attribute is selected and the same operation is repeated. The goal is to obtain, in the leaves, a set of cases all belonging to the same assignment class (in the example, "buy" or "not buy"). The procedure stops when the maximum depth set for the tree is reached or when the obtained subclass has very low variability so that further splitting does not produce a more homogeneous set.

The selection of attributes allowing an effective model with a reduced number of steps is therefore a very delicate phase. The example shown in the figure considers a **general tree**, where each attribute can assume any (but discrete) number of values. However, this condition is not general, and an attribute may have a fixed number of values (if only two values, it is called a **binary tree**). The greater the number of values, the more complex the model construction. In the case of continuous attributes, thresholds can be applied to convert them into discrete attributes. Moreover, the transition from one level to the next can depend on the evaluation of a single attribute (**univariate tree**) or on the combination of multiple attributes' values (**multivariate tree**).

As mentioned earlier, the obtained model can be evaluated based on its ability to classify elements of the test set. Depending on the choices made (order of selected attributes at each step), different decision tree models can be constructed for a dataset. The evaluation allows selecting the one best able to represent the

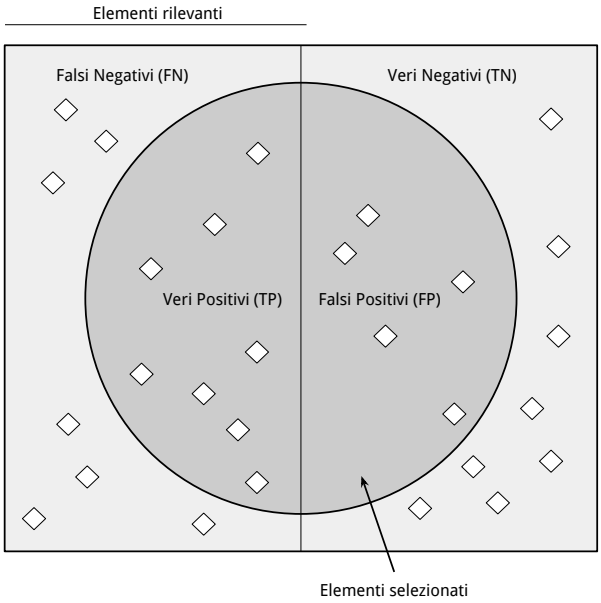


Figure 7.11: Illustration of possible classification outcomes: false positives, false negatives, true positives, and true negatives

considered dataset.

In this regard, the role of the *test set* is fundamental. Indeed, for each element, through the labeling operation, its correct classification is known. Returning to the television purchase example, for each individual, it is indicated whether they intend to purchase the television or not. A decision tree is therefore perfect when it can classify every element in the same class as labeled. Referring to the example in Figure 7.10, for each test set element under 30 years old, purchase of the television should not be predicted, otherwise it would conflict with the labeling information.

In reality, discrepancies always exist, and these are calculated starting from the analysis of so-called *false positives* and *false negatives* (see Figure 7.11). A false negative occurs when an individual is not classified as belonging to a class but should actually belong. For example, the tree predicts that a user does not belong to the class of buyers who will purchase the television, while in reality, in the test set, this individual is labeled as a buyer. Conversely, a false positive occurs if an individual is associated with the class of television buyers but will not actually buy the television.

Similarly, the so-called true positives and true negatives can be defined, which, unlike the previous cases, indicate when an individual is correctly inserted or not inserted into a class.

Once the total numbers of: (i) false positives (FP), (ii) false negatives (FN), (iii) true positives (TP), and (iv) true negatives (TN) for the entire test set are

calculated, *Precision* and *Recall* are defined.

Precision is the percentage of elements correctly classified among all those classified as belonging to a class:

$$Precision = \frac{TP}{TP + FP}$$

Intuitively, Precision indicates how accurately a decision tree inserts individuals into a class. For example, a Precision of 66% indicates that in two-thirds of the cases, an individual identified as a potential television buyer actually becomes one.

Recall indicates how many elements the classification algorithm was able to associate with the class compared to those that ideally constitute the class:

$$Recall = \frac{TP}{TP + FN}$$

Referring to the example above, a Recall of 50% indicates that the elements the decision tree inserted in the group of potential television buyers constitute only half of the actual set of people who will really buy the television.

7.2.4 Clustering

Clustering, like classification, allows grouping a set of elements based on their characteristics by assigning them a class membership. The main difference between the two approaches is that clustering does not require knowing the training set class memberships in advance but finds the classifications and sometimes even their number automatically based on the similarity of the attribute values of the dataset elements. The clustering result is thus a set of clusters (groups) composed of elements satisfying the following conditions: (i) similarity among elements within the same cluster is maximized; (ii) similarity between elements of different clusters is minimized.

Clustering methods are applied in various fields such as social sciences, image recognition, and marketing analysis. They are based on the human brain's ability to group elements based on affinity. For example, a set of customers could be grouped via clustering based on their product purchasing behavior.

To calculate the distance between elements in a set, the attributes describing the elements must be numeric. The main requirements by which clustering algorithms are evaluated are: (i) flexibility — ability to analyze categorical as well as numerical elements; (ii) robustness — stability of results in the presence of slight variations in attribute values; (iii) efficiency — time required to obtain the dataset partition into clusters.

The most widespread clustering algorithm is the *k-means* algorithm, an unsupervised technique. It requires (i) prior knowledge of the number k of clusters to identify, (ii) a similarity function indicating the closeness between two elements. The similarity function usually depends, through weighted sums, on the attributes defining an individual in the analyzed set. Thinking of supermarket customers,

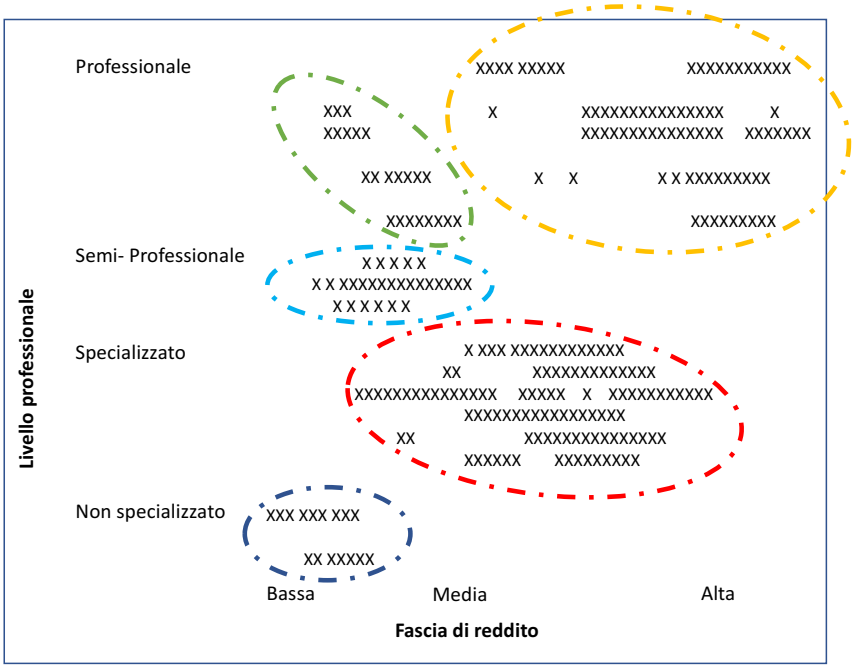


Figure 7.12: Example of clustering ($k = 5$) used for market segmentation

their similarity could depend on age, geographic area of origin, and profession. By calculating the difference between these dimensions' values and weighting their importance, one can quantify how similar one individual is to another.

The similarity function is applied iteratively to the set for which clusters are to be identified. The approach starts by identifying the centroids' positions, one for each cluster, representing the point with coordinates corresponding to the mean attribute values for the elements in a set. Initially, centroids are positioned randomly. Dataset elements are then assigned to a cluster based on their distance from each centroid. For each identified set, centroid positions are recalculated, and element assignments are repeated. The algorithm stops when the system converges (centroids have a stable position and the assigned elements to sets do not change from one iteration to the next).

An example of clustering is shown in Figure 7.12, where clustering is used for market segmentation, i.e., dividing customers into "social groups" that constitute market segments based on geographic, socio-demographic, or behavioral characteristics. In the example, customers are grouped based on two attributes: their income bracket (low, medium, high) and their professional level (unskilled, skilled,

semi-professional, and professional). Starting from the attribute distribution, five clusters are identified, each representing a segment. It is important to note that clusters are not assigned a clear category since they are not predefined, but each set represents elements sharing similar attribute values.

Clustering algorithms are classified based on element assignment to clusters. Specifically, an algorithm is **exclusive** if it assigns each dataset element to only one cluster, while it is **overlapping** if the same element can be assigned to multiple classes. **Fuzzy** methods are overlapping methods in which assignment of an element to each class is associated with a weight varying between 0 and 1. Methods are also classified as **complete** if every element belongs to at least one class and **partial** if some elements may not be assigned to any class.

7.2.5 Examples on CRM

Data mining techniques are used by analytical CRM to analyze customer actions and behavior and find significant patterns. For example, association rules are used to analyze different customers' purchases and find products that are often bought together. In particular, rules of the form $X \Rightarrow Y$ can be used to recommend Y to all customers who buy X. For example, consider the recommendation services offered by Amazon: when we visit a book's page, the site, under the section "Customers who bought this item also bought," suggests other books that other users have purchased together with the book we are viewing. This is the result of association rules that link the book we considered (X) to other books frequently bought together (Y). Association rules allow companies to be proactive and anticipate customer desires.

Classification methods (for example, decision trees) allow us, starting from a set of objects, their features, and their classification, to find the method and criteria through which to automatically assign objects to classes. These techniques have often been used in the insurance sector: given the clients, their history, and the risk category, methods have been defined to classify new clients into probable risk categories and determine an associated premium. Classification criteria in this sector include age, area of residence, and the type of insured vehicle. Alternatively, a company manually classifies clients based on their history into different categories such as:

- *Platinum*: reliable users, not price sensitive, willing to try new products, loyal.
- *Gold*: users who make many purchases but look for discounts, buy from multiple suppliers, and are not loyal.
- *Iron*: users who generate low business volume or are inconsistent in purchases.
- *Lead*: demanding users who require special attention but do not show loyalty.

By applying classification methods, it is possible to find customer characteristics to use as classification criteria in order to assign categories a priori and understand which marketing strategies to adopt.

Clustering is used to segment customers and form groups of similar customers. Consider a telephone provider that wants to segment its customer base by categories

of service usage. If customers can be classified by usage, the company can offer more attractive subscriptions to its clients. Variables are given by the use of the following services: Multiple lines, Voice mail, Internet, Caller ID visualization, Call waiting, Call forwarding, 3-way calling, Electronic billing. Applying clustering techniques leads to identifying three clusters:

- *Cluster 1*: customers who spend a lot and buy many services.
- *Cluster 2*: customers who tend to spend moderately on calling services.
- *Cluster 3*: customers who spend very little and buy few services.

Segmentation provides information for marketing campaigns and the offer of advanced services tailored for a specific category of customers.

7.3 Process mining

Business processes are a significant part of an organization's information system and provide a structured approach to achieve business objectives efficiently. However, it is necessary to ensure that processes are correctly modeled, guaranteeing the effective execution of business activities. Moreover, it is essential to monitor that the way activities are actually carried out within the organization reflects the behavior expected and formalized through process models.

Process Mining [29, 30] is an approach that allows the analysis of business processes based on monitoring data generated by their execution, helping organizations understand how processes are actually performed and highlighting any deviations from the original design. Process Mining discovers inefficiencies, bottlenecks, deviations, and opportunities for improvement in workflows, enabling data-driven decision-making for process optimization.

Process Mining is used to discover, monitor, and improve real processes. The essential ingredients of Process Mining are:

- the *monitoring data* related to the execution of activities composing a business process;
- the *process model* describing the order in which activities should be executed.

Based on these inputs, analyses can be performed oriented both to verify the quality of business processes (why are some activities delayed? what are the bottlenecks? which resources are over-utilized?) and the compliance of their execution (which activities are skipped? which exceptions are not properly handled?).

Process Mining can support all phases of a process's lifecycle. Process mining techniques can be classified as follows (Fig. 7.13):

- **Process Discovery**: a process model is generated from an event log without using any a priori information;

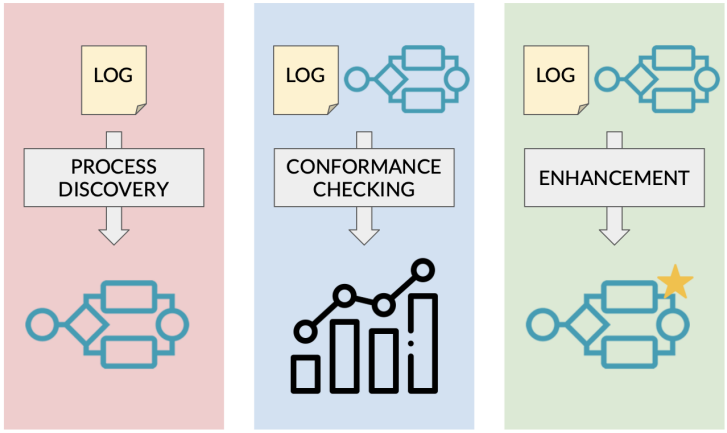


Figure 7.13: Main Process Mining techniques

- **Conformance checking**: a model is compared with an event log to verify whether reality conforms to the model (and vice versa);
- **Enhancement**: an existing model can be improved or extended using information provided by an event log.

7.3.1 Event Logs

One of the fundamental elements of Process Mining consists of event logs generated by processes. An event log is a collection of data recording individual events or activities occurring during the execution of a business process. Each event in the log is characterized by some fundamental information:

- Case ID: a unique identifier for the process instance;
- Activity: the name of the process activity to which the event refers;
- Timestamp: the date and time when the event occurred. It usually represents the moment the activity started but may also record the completion time.

Activities require time to be executed. The event can also record the status of each activity (start, completion, suspension, resumption, cancellation). Additionally, each event can also record information about the resources involved in carrying out the activity (personnel, system, manager, etc.) and additional information useful for analysis. An example is shown in Figure 7.14, where the additional information indicates the involved customer (Customer).

A log can also be organized as a sequence of traces. A trace is the list of activities executed for a specific process instance (Case ID), ordered by execution timestamp. In this case, the log is a set of traces corresponding to different executions of the

Case	Activity	Timestamp	Resource	Customer
...	...	...	...	...
pizza-56	buy ingredients (bi)	18:10	Stefano	Valentina
pizza-57	buy ingredients (bi)	18:12	Stefano	Giulia
pizza-57	create base (cb)	18:16	Mario	Giulia
pizza-56	create base (cb)	18:19	Mario	Valentina
pizza-57	add tomato (at)	18:21	Mario	Giulia
pizza-57	add cheese (ac)	18:27	Mario	Giulia
pizza-56	add cheese (ac)	18:34	Mario	Valentina
pizza-56	add tomato (at)	18:44	Mario	Valentina
pizza-56	add salami (as)	18:45	Mario	Valentina
pizza-56	bake in oven (bo)	18:48	Stefano	Valentina
pizza-57	add salami (as)	18:50	Mario	Giulia

Figure 7.14: Example of an event log related to a pizzeria process

process. This representation is very convenient to group together similar traces (characterized by the same set of activities in the same order) and analyze the most common and rare traces.

7.3.2 Process Discovery

In the section on process identification, we analyzed existing approaches that allow the organization to identify the processes characterizing its activities and highlighted how this activity is very complex and requires a high expenditure of time and resources to be carried out. Process Discovery is a Process Mining approach that supports the identification of processes through automated tools. The algorithms supporting Process Discovery allow reconstructing a process model exclusively from an event log, without relying on predefined models or hypotheses.

Process Discovery focuses on the control flow to transform the event log into the process representation considering only the activities and their sequence in the various traces. The resulting process model must allow all behaviors present in the log (each trace must be executable through the model).

Process Discovery starts from the analysis of traces in the event log from which it extracts the existing relations between pairs of activities. Possible relations are:

- Direct succession ($>$) – an activity is immediately followed by another ($x > y$) in at least one trace in the log;
- Causality ($\rightarrow$) – an activity can follow another but not vice versa ($x \rightarrow y$ if $x > y$ and $\sim (y > x)$);
- Parallel ($||$) – two activities are executed in no specific order ($x||y$ if $x > y$ and $y > x$);

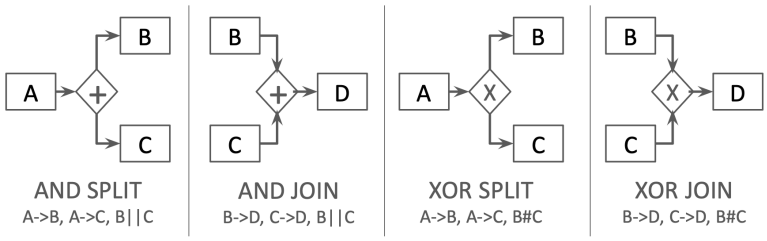


Figure 7.15: Process discovery: types of relations and BPMN gateways

- Choice (#) – the two activities are never executed in sequence ($x \# y$ if $\sim (x > y)$ and $\sim (y > x)$).

Without going into the details of Process Discovery algorithms, we can use the relations just described to translate them into the typical gateways introduced in BPMN notation, as shown in Figure 7.15.

7.3.3 Conformance Checking

Conformance checking is a process mining approach that consists in detecting any deviations between the intended behavior of a business process (expressed through a model) and its actual execution (recorded via event logs).

The comparison is performed via *replay* activity, which consists in attempting to execute the traces observed in the logs in relation to a business process. Different issues can be detected:

- deviations of log events (which traces cannot be replayed by the model?);
- deviations of the model (which activities are not conforming?);

Fitness is defined as a measure of correspondence between model and log.

Conformance checking can thus be used for two main objectives: i) verify whether actors within the organization behave as expected, and ii) evaluate the quality of process discovery by comparing different process models to find the one that best fits the event log.

Deviations are not always to be considered negative. They can be due to operators' reactions to some unexpected problems. In these cases, a deviation could prevent serious damage. Moreover, deviations can also indicate inadequacy of the process model. Users might find a better way of doing things than prescribed. It is therefore important both to detect deviations and to study and learn from them.

7.3.4 Process Improvement

Process mining can also be used to analyze process behavior in detail. From a process model and its event logs, it is, for example, possible to analyze how frequently the activities composing the process are executed, which are the most frequent

paths, how many execution variants exist, and what the average, maximum, and minimum execution times of different activities are. All this information can be very useful to find inefficiencies and possible improvement areas in business processes.

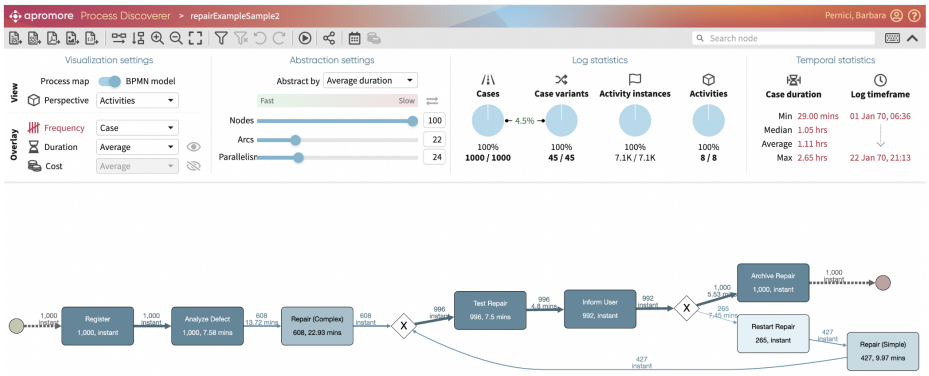


Figure 7.16: Process improvement: example of an analysis dashboard with Apromore

Figure 7.16 shows an example of a process analysis dashboard using the tool Apromore[1]. The process shown models the repair activities of machinery in a factory. The dashboard shows both the frequency and execution times of single activities as well as summary metrics related to the entire process. The most frequent paths are highlighted with thicker arrows between the activities composing the process.

7.4 Questions

- Illustrate the architecture and main properties of a Data Warehouse. Exemplify some problems related to data extraction and loading.
- Describe the components of a data warehousing system. Then briefly explain how companies can use the outputs of such an application.
- Illustrate classification algorithms and discuss their usefulness in analytical CRM.
- Illustrate the criteria for validating learning models.
- Describe the structure of logs in process mining and explain their usage.
- Illustrate the concept of conformance checking in process mining and discuss a simple example of a process model in BPMN with examples of logs with and without deviations.

[1] `https://apromore.com`

Bibliography and analytical index

Bibliography

[1] R.N. Anthony. *Planning and Control: a Framework for Analysis*. Cambridge MA: Harvard University Press, 1965.

[2] D. Ardagna, M.G. Fugini, B. Pernici, and P. Plebani. *Sistemi informativi basati su web*. Franco Angeli, Serie Sistemi Informativi, vol. 6, 2006.

[3] Jean-Philippe Aumasson. *Serious cryptography: a practical introduction to modern encryption*. No Starch Press, Inc, 2024.

[4] P. Baltzan and A. Phillips. *Business Driven Information Systems*. McGraw-Hill, 2019.

[5] Scott A. Bernard. *An Introduction To Enterprise Architecture: Third Edition*. AuthorHouse, 2012.

[6] Giampio Bracchi, Chiara Francalanci, and Gianmario Motta. *Sistemi Informativi d'Impresa*. McGraw Hill, 2010.

[7] Cinzia Cappiello, Pierluigi Plebani, and Monica Vitali. Business Process Modeling Notation (BPMN), 2015.

[8] Silvana Castano, Mariagrazia Fugini, Giancarlo Martella, and Pierangela Samarati. *Database Security*. Addison-Wesley & ACM Press, 1995.

[9] ENISA. The goal of cybersecurity certification under the cybersecurity act. https://www.enisa.europa.eu/topics/certification/cybersecurity-certification-framework, 2024.

[10] Lois Evans. Protecting information assets using ISO/IEC security standards. *Information Management*, 50(6):28, 2016.

[11] World Economic Forum. Global Risks Report 2024. https://www.weforum.org/publications/global-risks-report-2024/, 2024.

[12] Mariagrazia Fugini, Fabrizio Maio, and Pierluigi Plebani. *Sicurezza dei Sistemi Informatici*. Apogeo, 2001.

[13] Gerben Wierda. *Mastering ArchiMate - Edition 3.1*. R&A, 2021.

[14] Paul Grefen. *Beyond E-Business: Towards networked structures*. Routledge, 2016.

[15] Matthew Hayward. Quantum computing and shor's algorithm. *Sydney: Macquarie University Mathematics Department*, 1, 2008.

[16] ISO/IEC/IEEE. Iso/iec/ieee 42010:2011 - system and software engineering: architecture description. IEEE, December 2011.

[17] Marc Lankhorst. *Enterprise Architecture at Work: Modelling, Communication and Analysis*. Springer, 2016.

[18] Kenneth C. Laudon and Jane P. Laudon. *Management Information Systems: Managing the Digital Firm, 15th Edition*. Pearson, 2017.

[19] Maurizio Pighin and Anna Marzona. *Sistemi informativi aziendali: struttura e processi*. Pearson, 2011.

[20] Federico Pigni, Aurelio Ravarini, and Donatella Sciuto. *Sistemi per la gestione dell'informazione*. Apogeo, 2009.

[21] M. Porter. *Competitive Advantage: Creating and Sustaining Superior Performance*. Free Press, 1985.

[22] Pierangela Samarati and Ravi Sandhu. *Database Security Volume X: Status and prospects*. Springer, 2016.

[23] Signavio. BPMN editor, http://academic.signavio.com.

[24] Craig Van Slyke and France Belanger. *Information Systems for Business: An Experiential Approach*. John Wiley & Sons Inc, 2011.

[25] J.F. Sowa and J.A. Zachman. Extending and formalizing the information systems architecture. *IBM Systems Journal*, 31(3):590–616, 1992.

[26] William Stallings. ICAM: A foundation for trusted identities in cyberspace. *IT Professional*, 18(1):26–33, 2016.

[27] The Open Group. *The ArchiMate Enterprise Architecture Modeling Language, 3.0*. The Open Group, 2016.

[28] HA Madni Uppal, Memoona Javed, and MJ Arshad. An overview of intrusion detection system (IDS) along with its commonly used techniques and classifications. *International Journal of Computer Science and Telecommunications*, 5(2):20–24, 2014.

[29] Wil Van Der Aalst. *Data science in action*. Springer, 2016.

[30] Wil MP van der Aalst and Josep Carmona. *Process Mining Handbook*. Springer Nature, 2022.

[31] Carlo Vercellis. *Business Intelligence: modelli matematici e sistemi per le decisioni*. McGraw-Hill, 2006.

[32] Mathias Weske. *Business Process Management - Concepts, Languages, Architectures, 2nd Edition*. Springer, 2012.

[33] John A. Zachman. A framework for information systems architecture. *IBM Syst. J.*, 26(3):276–292, September 1987.

Index

Sales Force Automation (SFA), 73

accuratezza, 93
agente, 136
alberi di decisione, 93
albero di decisione binario, 94
albero di decisione generale, 94
albero di decisione multivariato, 94
albero di decisione univariato, 94
Algoritmi a chiave asimmetrica (o a chiave pubblica), 140
Algoritmi a chiave simmetrica (o a chiave segreta), 140
Anthony's Pyramid, 8
application logic - A, 62
application portfolio, 55
APS - Advanced Planning and Scheduling, 57
Archi, 31
ArchiMate, 29
as-is, 26
association rules, 92
attacchi, 137
autenticazione, 136, 149, 150
autenticità, 136

benefici, 188
BI, *see also* Business Intelligence
BPMN (Business Process Modeling Notation), 44
BPMS - Business Process Management Systems, 58
Business Intelligence, 55, 77
business process, 3, 43
business rules, 65
business service, 33

Certification Authority - CA, 148
certificato a chiave pubblica - PKC, 148
ciclo di vita, 192
classificazione, 93
cloning, 127
cloud computing, 132, 134
Cloud Computing deployment models, 133
cluster, 96
clustering, 96
community cloud, 134
components (elements) in ArchiMate, 32
confidence, 92
conformance checking, 100
costi, 188
COTS, 57, 63, 67, 188, 193
CRM, 57, 66, 69, 98
Customer Relationship Management, *see also* CRM
Cyberspace, 170

data access - D, 62
data mart, 81
data mining, 75, 90
data warehouse, 55, 72, 75, 78, 80
DBMS, 14, 57, 61
Denial of Service - DoS, 137
dice, 89
Dimensional Fact Model (DFM), 83
dimensioni di qualità, 187
Discretionary Access Control (DAC), 155
disponibilità, 136
drill-down, 87
DW, *see also* data warehouse

E-Business, 57
EB, *see also* E-Business
efficacia, 182
efficienza, 182

elasticity, 127
enhancement, 100
Enterprise Architecture - EA, 16, 23, 180
Enterprise Resource Planning, *see also* ERP
entropia, 94
ERP, 57, 63
ETL, 82, 89

falso negativo, 95
falso positivo, 95
FASMI, 80
fattibilità tecnica, 187
firewall, 161
firma digitale, 146
framework di Zachman, 23
funzione di somiglianza, 97
funzioni di hash, 145

gestione del sistema informativo, 179
gestione delle chiavi, 146

HOLAP, 85
Human Resource, 10
hybrid cloud, 134

IDS (intrusion Detection System, 164
indicatori, 182
Infrastructure as a Service, 133
integrità, 136
interpretabilità, 93

k-means, 96
Key Performance Indicator (KPI), 182

layer, 61
layers in ArchiMate, 29
legacy system, 62

Make-or-Buy, 57, 188
  Buy, 63, 188, 193, 194
  Make, 188
Mandatory Access Control (MAC), 155
Manifacturing Execution Systems, *see also* MES
MES, 57
metadata, 83
minacce, 136
MOLAP, 84
Multi Factor Authentication (MFA), 151

obiettivi, 186
OLAP (On-line Analytical Processing), 14, 78, 80
OLTP (On-line Transaction Processing), 14, 78
One Time Password (OTP), 140, 152
Outsourcing, 191
outsourcing, 187

partitioning, 127
pianificazione strategica, 181
Platform as a Service, 133
PLM (Product Lifecycle Management), 66
policy, 139
politiche di sicurezza, 154
Porter
  primary activities, 9
  support activities, 9
  value chain, 9
Precision, 96
presentation - P, 61
privacy, 136
private cloud, 134
process, 8
process discovery, 99
process mining, 99
progettazione delle soluzioni, 186
proprietà di sicurezza, 135
public cloud, 134
Public Key Infrastructure - PKI, 148

QoS - Quality of Service, 114
qualità
  affidabilità, 187
  disponibilità, 187
  maturità tecnologica, 187
  scalabilità, 179, 187
  sicurezza, 187

RACS - Reliable Array of Cloned Services, 127

RAPS - Reliable Array of Partitioned Services, 128
Recall, 96
relationships in ArchiMate, 32
requirements pull, 18, 21
resource, 6
REST - Representational state transfer, 115
reti Bayesiane, 93
reti neurali, 93
riservatezza, 136
robustezza, 93
ROLAP, 85
roll-up, 87
Rule Engine, 58

SaaS, 134
scalability, 126, 127
scalabilità, 93
SCM (Supply Chain Management), 66, 68
server farm, 127
service orchestration, 116
service registry, 115
services, 29, 58, 113
shared-disk, 127
shared-nothing, 127
Signavio, 47
single tiered, 121
sistema legacy, 187
SLA - Service Level Agreement, 114
slice, 89
snowflake schema, 85
SOA - Service Oriented Architecture, 113, 115
SOAP - Service Oriented Architecture Protocol, 115
Software as a Service, 133
stakeholder, 22
star schema, 85
studio di fattibilità, 181, 184
supervised learning, 90
support, 92

technology push, 18, 21
test set, 93
three tiered (3-Tiers), 122
tier, 121
to-be, 26
training set, 93
transaction, 14
two tiered (2-Tiers), 122

UIS - User Interface Systems, 58
unsupervised learning, 91
utility computing, 132

velocità, 93
vero negativo, 95
vero positivo, 95
violazioni, 137
virtual machine - VM, 130
virtualization, 129

Zero trust, 170